



PROFILER-ADAPTIVE DATA STAGING FOR LARGE-SCALE SATELLITE-IMAGE SEGMENTATION ON HPC SYSTEMS

Author

NABEEL AHSAN

Supervised by

DR. LEON VAN DER TORRE, University of Luxembourg
RICHARD ALBRECHT, co-supervisor

Reviewed by

DR. ANDREA BINSFIELD

Examined by

DR. LEON VAN DER TORRE
DR. ANDREA BINSFIELD

A Thesis submitted in fulfillment of requirements for the degree of
Master in High Performance Computing

Department of Computer Science
University of Luxembourg
2026

Declaration of Authorship

For single-authored work:

I hereby declare that I am the sole author of the work entitled.

Profiler-Adaptive Data Staging for Large-Scale Satellite-Image Segmentation on HPC Systems

and here enclosed, and that I have compiled it in my own words, that I have not used any other than the cited sources and aids, and that all parts of this work, which I have adopted from other sources, are acknowledged and designated as such. I also confirm that this work has not been submitted previously or elsewhere.

Luxembourg, 2026

Date

Signature

Nabeel Ahsan

First name and surname

Abstract

Deep learning training on high-performance computing (HPC) clusters is conventionally treated as a problem of GPU throughput, and optimisation effort is directed at the model. For semantic segmentation over remotely sensed imagery, however, the training corpus is typically stored as very large numbers of small image and mask files on a shared parallel filesystem, and the cost of reaching those files is borne once per sample. We show that for a representative archaeological site-detection workload - the detection of mounded settlement sites (*tells*) in the southern Mesopotamian floodplain [6] - the GPU is frequently idle while the data path is the binding constraint: across three architectures, time spent waiting on data exceeds forward-and-backward compute time by 8–20× under a default loader configuration, and two loader flags alone move GPU utilisation by 24–51 percentage points. Segmentation quality moves far less under the same change, and not consistently in one direction, which is itself informative: this workload’s headroom is in throughput, not accuracy. We introduce PADS-Loader (Profiler-Adaptive Data Staging), a drop-in replacement for the PyTorch `DataLoader` that profiles its own behaviour over a short warm-up window, classifies the dominant bottleneck from the measured timings, selects a staging policy - loose files, tar shards, or node-local scratch staging - and executes that policy with a prefetch depth derived from the measurements. Because the sample permutation is fixed at the start of each epoch, the identity of future file accesses is known exactly, so prefetching is exact rather than speculative. We evaluate PADS-Loader against an unmodified loader, a manually tuned loader, fixed tar-shard streaming, and a full pre-staging upper bound, on three segmentation architectures spanning a CNN encoder–decoder, an attention-augmented CNN, and a transformer. The staging mechanism itself delivers on that promise when actually invoked: tar shards alone beat an unmodified loader on GPU utilisation for every architecture tested, and a full pre-staging ceiling reaches the highest GPU utilisation of any configuration tested for two of three architectures. PADS-Loader’s adaptive policy selection, however, activated staging in only a small minority of the configurations where the fixed-policy experiments showed that staging could provide a measurable benefit. This behaviour was traced to a specific and quantified defect in the profiler’s warm-up measurement. One-time GPU initialisation costs dominated the short averaging window, inflating the measured GPU compute time and systematically understating the true data-to-compute time ratio by approximately an order of magnitude. The resulting conservative policy selection therefore reflects a limitation of the profiling procedure rather than a failure of the staging mechanism itself. . A validated correction, discarding warm-up batches before measuring, recovers full-epoch measurement accuracy at negligible added cost. Alongside the positive throughput claim we make a negative accuracy claim: a pipeline optimisation is only sound if segmentation quality is preserved, and we therefore report intersection-over-union against a reference reproduction throughout.

Keywords: Data Staging; I/O Bottlenecks; Semantic Segmentation; Remote Sensing; High-Performance Computing

Plain Language Summary

Semantic segmentation – assigning a label to every pixel of an image rather than to the image as a whole – has become a practical tool in archaeology. This thesis works with *tells*: mounds in southern Iraq built up from the accumulated debris of settlements occupied over long periods, and distinctive enough in satellite imagery that a neural network can be trained to trace their outlines. There are thousands of them across the survey region, so an automated first pass is worth a great deal to archaeologists deciding where to spend limited excavation effort.

Training such a network on a computing cluster is conventionally treated as a problem of keeping the GPU busy: enable mixed precision, raise the batch size, optimise the model. This thesis measures where the time actually goes, and the GPU turns out not to be the constraint. The training set is roughly ten thousand small files on a parallel filesystem shared with every other job on the cluster. Loading one sample means a separate open, stat, read and close for the image and again for its mask, and the cost of those metadata operations is close to constant no matter how few bytes are wanted. Measured across three architectures, time spent waiting for data exceeded time spent computing on it by a factor of eight to twenty; changing two loader settings moved GPU utilisation by 24 to 51 percentage points.

The contribution is a replacement data loader that profiles its own behaviour during a short warm-up, identifies which stage dominates, and selects one of three strategies: read files individually as before, read them from tar archives so that many small reads become few large ones, or copy them to fast node-local disk before they are needed. The third is possible because the order in which samples will be used is fixed at the start of each pass through the data. The loader therefore knows exactly which files it will need and when, which makes preparing them ahead of time a scheduling problem rather than a prediction problem – the distinction that separates this from an ordinary cache.

The evaluation produced one positive result and one negative one. The mechanisms themselves work: reading from archives beat reading individual files on every architecture tested, and copying the dataset to local disk in advance gave the highest utilisation of any configuration for two of the three. The automatic selection did not work: it almost always chose the plain strategy even where a better one was available. The fault lay in the measurement rather than the decision rule. The profiling window was only eight batches long – short enough that one-off GPU initialisation dominated the average and made computation appear six to ten times slower than it really is, which in turn made the data path look adequate by comparison. Discarding the first few batches before measuring recovers an accurate reading at no meaningful cost.

The wider point is not specific to archaeology. On shared clusters the input pipeline is frequently the limiting factor rather than the processor, and any system that configures itself from its own measurements is only as reliable as the window those measurements are taken over.

Acknowledgements

The experiments presented in this work were carried out using the HPC facilities of the University of Luxembourg [38]: <https://hpc.uni.lu>.

Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
BN	Batch Normalisation
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DALI	NVIDIA Data Loading Library
DL	Deep Learning
FN	False Negative
FP	False Positive
GPFS	General Parallel File System
GPU	Graphics Processing Unit
HBM	High-Bandwidth Memory
HPC	High-Performance Computing
IoU	Intersection over Union
MCC	Matthews Correlation Coefficient
MLP	Multi-Layer Perceptron
PADS-Loader	Profiler-Adaptive Data Staging Loader
POSIX	Portable Operating System Interface
RAM	Random Access Memory
RNG	Random Number Generator
RQ	Research Question
SRAM	Static Random Access Memory
TN	True Negative
TP	True Positive
VRAM	Video Random Access Memory

Contents

Declaration of Authorship	ii
Abstract	iv
Plain Language Summary	vi
Acknowledgements	viii
Abbreviations	x
List of Figures	xiv
List of Tables	xv
1 Introduction	1
1.1 Background and Motivation	1
1.2 The problem	1
1.3 Research question	2
1.4 Contributions	3
1.5 Thesis structure	3
2 Related Work	3
2.1 Semantic segmentation architectures	4
2.2 Deep learning for archaeological prospection	4
2.3 Input pipelines and I/O for large-scale training	5
3 Materials and Datasets	6
3.1 Overview	6
3.2 Dataset: Casini et al. (2023)	6
3.2.1 Provenance and construction	6
3.2.2 Class imbalance	7
3.2.3 Evaluation metrics	8
3.2.4 Reference results	9
3.3 The dataset as an I/O workload	9
3.4 Models	10
3.4.1 MA-Net	10
3.4.2 DeepLabV3+	10
3.4.3 SegFormer	11
3.4.4 Measured characteristics (Bing 1k, default loader)	11
3.5 Hardware and software environment	11
4 Methodology	12
4.1 Motivation: when the DataLoader becomes the bottleneck	12
4.2 PADS-Loader: system overview	12
4.2.1 Stage 1 - Warm-up profiling	12
4.2.2 Stage 2 - Bottleneck classification	13
4.2.3 Stage 3 - Policy selection	14
4.2.4 Stage 4 - Predictive staging	14
4.3 Metrics captured	15
4.4 Evaluation plan	16
4.5 Experimental protocol	17

5	Results	17
5.1	RQ1: When does the DataLoader become the bottleneck?	17
5.2	RQ2: Do tar shards reduce many-small-file overhead?	18
5.3	RQ3: Does predictive staging outperform fixed streaming?	19
5.4	RQ4: Can prefetch depth be auto-tuned?	20
5.5	Profiling overhead	22
5.6	Why the adaptive layer stayed conservative: a profiler measurement defect	22
5.7	Accuracy is preserved	24
6	Ablation Study	25
6.1	Component ablation	25
6.2	Scratch capacity	26
6.3	Classifier threshold sensitivity	27
6.4	Shuffling randomness	27
7	Conclusions	28
7.1	Summary of contributions	28
7.2	Answers to the research questions	29
7.3	Closing remarks	30
8	Limitations and Future Work	30
8.1	Limitations	30
8.1.1	Scope of the claims	30
8.1.2	A single filesystem and cluster	30
8.1.3	Uncontrolled contention and unestimated variance	30
8.1.4	One corpus	30
8.1.5	Single-node scope	31
8.1.6	Rule-based classification	31
8.2	Future work	31
8.2.1	Decode offload	31
8.2.2	Cross-job coordination	31
8.2.3	Online rather than one-shot adaptation	31
8.2.4	Revisiting segmentation accuracy	32
8.2.5	Transfer beyond archaeology	32
9	Discussion	32
9.1	Regional transferability	32
9.1.1	Scope of transfer	32
9.1.2	Sources of domain shift	32
9.1.3	Adaptation through transfer learning	32
9.1.4	Annotation as the binding constraint	33
9.2	Transferability across archaeological feature classes	33
9.2.1	Scope across feature classes	33
9.2.2	Visibility, scale, and difficulty	33
9.3	Reliability and interpretation	33
9.3.1	Interpretation of aggregate IoU	34
9.3.2	Per-site error distribution	34
9.3.3	Reliability of systems measurements	34
9.4	Verification strategy	34
9.4.1	Held-out evaluation	34
9.4.2	Repeated evaluation	34
9.4.3	Independent reproduction	34
9.4.4	Limits of verification	35
9.5	Error sources and feature confusion	35
9.5.1	False-positive detections	35
9.5.2	False-negative detections	35
9.5.3	Annotation ambiguity	35

9.5.4	Role of expert review	35
9.6	Archaeological contribution	36
A	Reproduction of the Reference Configuration	40
A.1	Environment	40
A.2	Faithfulness of the port	40
A.3	Result and the batch-size constraint	40
A.4	Closing the gap: full-protocol replication on the cluster	41
A.5	Metric reduction	41
A.6	Error structure	42
A.7	Accuracy guardrail: segmentation quality across data-path policies	42
A.8	Qualitative output	43
B	Pipeline Defects Found and Corrected	44
B.1	Staging mechanism: three defects, all corrected	44
B.1.1	Policy selected after the datasets it selects were constructed	44
B.1.2	No archive the shard reader could open	44
B.1.3	Staging silently degraded to a plain read	45
B.2	Issues bearing on segmentation accuracy	45
B.3	A measurement consideration: profiling perturbs the training RNG	45
B.4	Further issues found during the evaluation campaign	45
B.5	Implication for the results reported in this thesis	47

List of Figures

Introduction	1
1 Example predictions on held-out test tiles	2
2 Scope and construction of the segmentation dataset	7
3 Architectural paths of the three evaluated models, summarised from the primary architecture papers [14, 10, 42].	10
4 GPU utilisation, RQ3 methods 3–5, all three architectures.	19
5 GPU utilisation across the prefetch-depth sweep	20
6 Measured ratio against the full-epoch ground truth	24
7 GPU utilisation by policy component, all three architectures.	26
8 GPU utilisation vs. scratch-capacity budget	27
9 Measured ratio vs. decision threshold	28

List of Tables

Materials and Datasets	6
1 Key properties of the dataset and the Bing 1k variant used throughout the primary evaluation [6].	8
2 IoU scores reported by Casini et al.	9
3 Baseline characteristics, default DataLoader	11
4 Characteristics with tuned loader parameters	11
Methodology	12
5 Signals recorded during the warm-up window.	13
6 Bottleneck classification rules.	13
7 The three staging policies.	14
8 Evaluation methods and the contrast each provides.	16
Results	17
9 GPU utilisation, default vs. tuned DataLoader	18
10 Per-stage timing breakdown by architecture	18
11 Filesystem operations per batch, loose policy	19
12 End-to-end comparison, evaluation methods 3–5	19
13 GPU utilisation at fixed prefetch depths	20
14 Auto-tuned prefetch depth against measured staging time	21
15 Cost of the profiling window	22
16 8-batch probe vs. full-epoch measurement	22
17 Measured ratio by post-discard window size	23
Ablation Study	25
18 Component ablation: GPU utilisation by policy component	25
19 Behaviour as scratch capacity is constrained	26
20 Sensitivity of the policy decision to the threshold	28
Reproduction of the Reference Configuration	40
21 Full-protocol replica results against the reference	41
22 Effect of the IoU aggregation convention	42
23 Per-image IoU by site area in the crop	42
24 Segmentation quality by data-path policy	43
Pipeline Defects Found and Corrected	44
25 Issues bearing on segmentation accuracy	46

1 Introduction

1.1 Background and Motivation

Remote sensing has become a central instrument of archaeological prospection. Where a field survey of a large region once required years of systematic ground inspection, openly available satellite imagery now permits candidate sites to be identified from a desk, and the resulting shortlist to direct where limited excavation effort is spent [6, 25]. The southern Mesopotamian floodplain is a canonical setting for this: it contains thousands of mounded settlement sites, locally called *tells*, formed by the accumulated building levels and occupation debris of communities that remained in one place over long periods [40]. Their colour and shape make them visible from above, and that visibility is what makes automated detection plausible at all.

Semantic segmentation - assigning a class label to every pixel - is well matched to this task, because archaeologists need the outline of a site rather than merely its presence. Formally, let an RGB tile be $x \in \mathbb{R}^{H \times W \times 3}$ and its binary annotation be $y \in \{0, 1\}^{H \times W}$. A segmentation network f_θ with parameters θ produces logits $z = f_\theta(x) \in \mathbb{R}^{H \times W}$ and probabilities $p_{ij} = \sigma(z_{ij})$. Training estimates

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(f_\theta(x_n), y_n), \quad (1)$$

after which $\hat{y}_{ij} = \mathbb{I}[p_{ij} \geq \tau]$ gives the predicted mask at threshold τ . This notation separates the model objective $\mathcal{L}$ from the systems objective studied here: reducing data-path time while holding the learned segmentation quality fixed. Casini et al. [6] fine-tuned pretrained segmentation networks on a corpus of roughly five thousand ground-truthed sites and reported intersection-over-union (IoU) in the range of 74–83%, with site-level detection accuracy near 80% once a domain expert had adjudicated the predictions. That work also established the framing that this thesis inherits: the model is a component in a human–AI workflow, not a replacement for the archaeologist.

The practical form of the task is illustrated by held-out predictions (Figure 1), using the reproduction checkpoints of Appendix A on held-out tiles. Where a mound is tonally distinct from the surrounding land, all three architectures recover its outline closely and agree with one another; performance falls away on small sites, and Section A.8 characterises that dependence in full.

The present work takes that workload as given and asks a different question. Training such models on a shared HPC cluster is conventionally treated as a problem of GPU throughput: the model is profiled, the batch size is tuned, mixed precision is enabled, and the data path is assumed to keep up. The corpus consists of thousands of individually stored image and mask files on a shared parallel filesystem, and each training step issues an independent `open/stat/read/close` sequence per sample against a filesystem optimised for large sequential transfers rather than for many-small-file metadata operations.

1.2 The problem

Three properties of this setting, taken together, define the problem addressed here.

The bottleneck is real and it is not the model. Profiling the baseline pipeline across three architectures shows a ratio of data-path time to compute time large enough that the loader is a plausible constraint rather than a rounding error, and the effect is most pronounced for the architecture with the lowest per-step compute - precisely the configuration in which a fast model exposes a slow pipeline.

The bottleneck moves, but the configuration does not. `num_workers`, `prefetch_factor` and `pin_memory` are fixed at loader construction and never revisited. On a shared filesystem, observed read latency is not a property of the dataset alone but of whatever else is running on the cluster, and therefore varies over the lifetime of a single job. A statically configured loader cannot respond to a bottleneck that shifts underneath it.

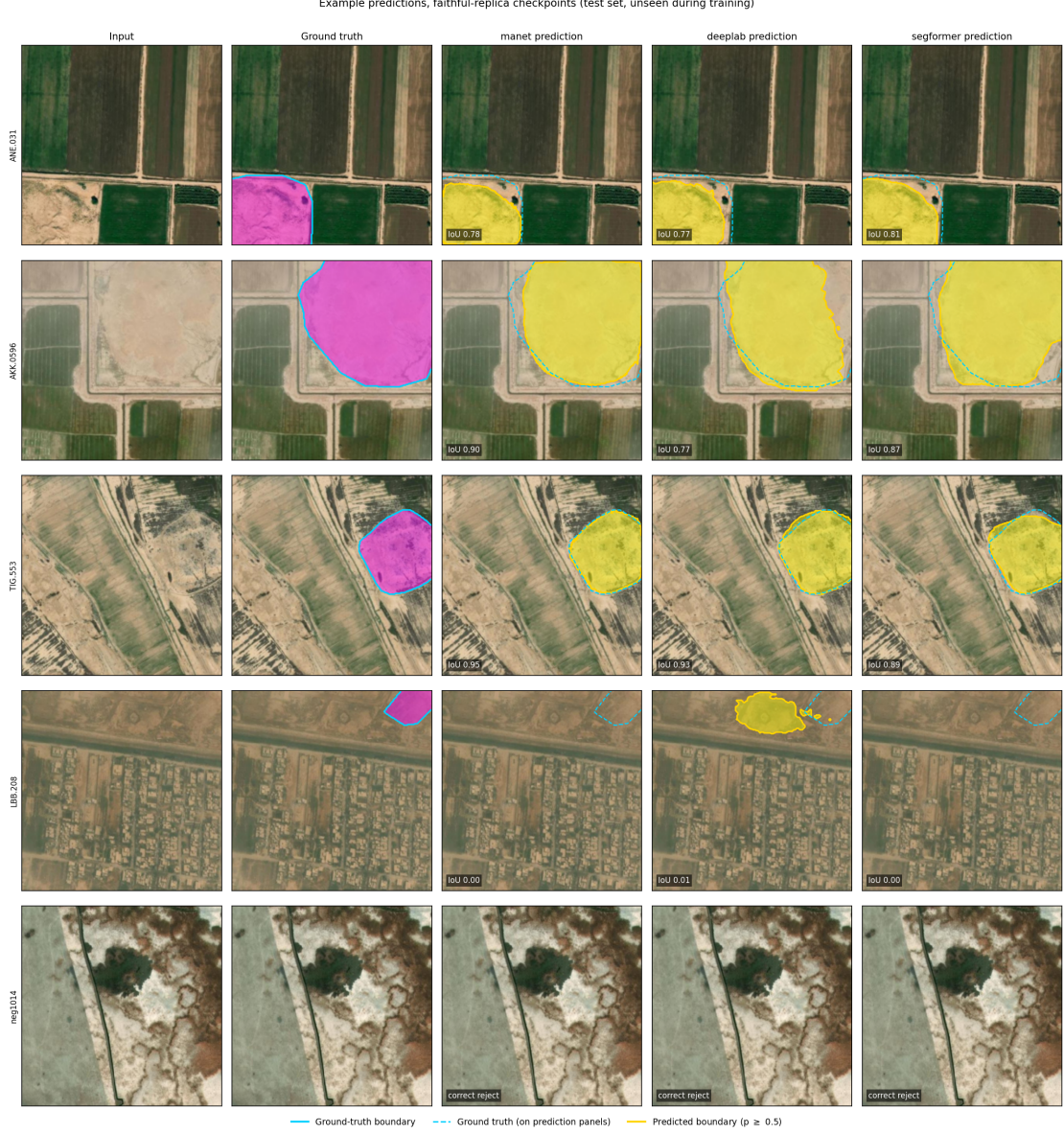


Figure 1: The segmentation task. Columns: input satellite imagery; the archaeologist’s annotated outline (filled, solid boundary); and each architecture’s predicted outline (filled, solid boundary) with the annotation repeated as a dashed line for comparison. Rows one to three are representative of the regime where all three architectures agree; row four is a characteristic failure on a small site; row five is a tile containing no site.

Future accesses are known exactly, not merely predictable. Once the epoch permutation is drawn, the sequence of samples - and hence the sequence of files - is fully determined. Prefetching in this setting need not be speculative. This is the property that the method exploits, and it is what separates an exact staging schedule from a heuristic cache.

1.3 Research question

The thesis addresses one question:

Can a profiler-adaptive **DataLoader** that combines bottleneck classification, tar-shard staging, and predictive prefetching improve GPU utilisation and tile throughput for large-scale satellite-image segmentation on HPC systems?

which decomposes into four sub-questions, each answered in this work:

RQ1. When does the `DataLoader` become the bottleneck on a shared parallel filesystem?

RQ2. Do tar shards reduce many-small-file overhead?

RQ3. Does predictive scratch staging outperform fixed shard streaming?

RQ4. Can prefetch depth be auto-tuned from measured timings?

1.4 Contributions

1. **A characterisation of the workload as an I/O problem.** We describe the segmentation corpus not only by its class balance and evaluation metrics, as is conventional, but by its file count, size distribution and access pattern - the characterisation that motivates staging and that the segmentation literature usually leaves implicit (Section 3).
2. **PADS-Loader, a profiler-adaptive data-staging loader.** A drop-in `DataLoader` replacement that measures T_{data} , T_{decode} , T_{h2d} and T_{gpu} over a bounded warm-up window, classifies the dominant bottleneck by an inspectable rule set, selects among loose, shard and stage policies, and derives its prefetch depth from the measured staging and consumption times rather than from a user-supplied constant (Section 4).
3. **An evaluation that isolates each mechanism.** PADS-Loader is compared against an unmodified loader, a manually tuned loader, fixed tar-shard streaming, and a full pre-staging upper bound, so that the benefit of sharding is separated from the benefit of staging, and the benefit of staging from the benefit of *predicting* what to stage (Section 5).
4. **A reproduction of the reference segmentation results, and an account of what it takes to obtain them.** We reproduce the reference configuration of Casini et al. [6] and document the pipeline defects and environmental sensitivities encountered in doing so, including a case in which reduced batch size - forced by GPU memory - degrades IoU substantially through its effect on batch normalisation statistics. This diagnosis is then tested directly rather than left as an inference. The experiment was repeated using the reference study’s original training protocol, including its batch size, epoch budget, and final-epoch weighting. Under these conditions, the reproduced results close the remaining accuracy gap to within 1–1.4 percentage points of the published values across all three architectures evaluated in this thesis (Appendix A).

1.5 Thesis structure

Section 2 reviews segmentation architectures, deep learning for archaeological prospection, and prior work on I/O for large-scale training. Section 3 describes the dataset in both of its characterisations and the three model architectures. Section 4 presents PADS-Loader. Section 5 reports the evaluation, and Section 6 isolates the contribution of individual components. Section 7 concludes, and Section 8 discusses limitations and further work.

2 Related Work

This chapter reviews three bodies of literature that provide the foundation for this thesis: semantic segmentation architectures (Section 2.1), deep learning for archaeological prospection (Section 2.2), and input-pipeline performance in large-scale deep learning (Section 2.3). The first two establish the application and computational workload considered in this thesis, while the third examines the input-pipeline performance problem and establishes the research gap addressed by PADS-Loader.

2.1 Semantic segmentation architectures

Semantic segmentation assigns a class label to every image pixel [30, 10]. The modern formulation builds on fully convolutional encoder-decoder networks, of which U-Net [30] is the canonical instance: a contracting path captures context, an expanding path recovers spatial resolution, and skip connections carry high-frequency detail across the bottleneck. U-Net was introduced for biomedical imaging under severe data scarcity, which is one reason it transferred well to remote sensing, where annotated corpora are similarly small [30, 6].

Two families extend this design in directions relevant here. The first enlarges the effective receptive field without sacrificing resolution. DeepLabV3+ [10] combines atrous (dilated) convolution and atrous spatial pyramid pooling with a decoder module that recovers object boundaries, capturing multi-scale context at high capacity. The second inserts attention into the encoder-decoder. MA-Net [14] augments a U-Net-like structure with self-attention in the latent space, allowing the network to weight latent features by content - an idea imported from the transformer literature [39]. Developed for liver and tumour segmentation, MA-Net has since been applied to remote sensing, and it is the architecture selected as best-performing by Casini et al. [6], which is why it is the reproduction baseline here.

A third family replaces the convolutional encoder entirely. SegFormer [42] pairs a hierarchical, positional-encoding-free transformer encoder with a lightweight all-MLP decoder, achieving competitive accuracy at markedly lower per-step compute than heavy CNN decoders. For the purposes of this thesis that low compute cost is not incidental: it makes SegFormer the model in which data-loading stalls are most prevalent, since a model that computes quickly spends proportionally more of its time waiting.

Encoder choice is largely orthogonal to decoder architecture. EfficientNet [33] obtains strong accuracy-per-parameter through compound scaling of depth, width and resolution, and EfficientNet-B3 is used throughout this work as the encoder for both CNN decoders, following the reference study. All architectures are instantiated through `segmentation_models_pytorch` [18], which allows encoder and decoder to be selected independently and supplies the loss functions used here.

2.2 Deep learning for archaeological prospection

Archaeology is what Bickler terms a “slow data” field: annotated examples accrue through fieldwork, not through scraping, and corpora are correspondingly small. This characterisation and its implications for archaeological machine learning are discussed explicitly by Bickler [4]. The standard response is transfer learning - initialising from a model pretrained on a large general dataset such as ImageNet [13] and fine-tuning on the specific task.

Applications divide by output granularity. Tile classification labels a map tile as containing a site or not; object detection predicts bounding boxes; semantic segmentation predicts a shape. Orengo et al. [25] applied machine-learning classification to multisensor and multitemporal satellite data for mound detection, working at the level of per-pixel classification over large regions. Trier et al. [34] used deep networks on airborne laser scanning data for semi-automatic mapping of archaeological topography, and Guyot et al. [17] combined detection and segmentation of archaeological structures from LiDAR. Verschoof-van der Vaart and Lambers [37] report on applying automated object detection within archaeological practice, emphasising the integration question rather than the accuracy question alone. A closely related strand uses object detection rather than segmentation to localise individual features: Caspari and Crespo [8] train a CNN to detect early Iron Age burial mounds in the Eurasian steppe, and Chen et al. [9] adapt off-the-shelf object detectors to find stone kurgans in the Altai Mountains; both report that the model narrows the area a human surveyor must inspect rather than replacing survey outright. Guyot et al. illustrate the same architectural shift within a single research line: an earlier Random Forest classifier over multi-scale LiDAR-derived topographic features [16] is followed, three years later, by a deep-learning detection-and-segmentation pipeline on the same class of data [17].

The Mesopotamian floodplain dataset used in this thesis originates from a specific research programme at the University of Bologna, and that programme’s own publication history illustrates how this literature accumulates. Rocchetti et al. [29] first assessed the feasibility of a deep-learning model for discovering new sites in this floodplain; Casini et al. [7] followed with

the preliminary semantic-segmentation study that this thesis reproduces (Appendix A); Casini et al. [6] then added the human-in-the-loop adjudication step discussed below; and, most recently, Pistola et al. [27] retrained the same underlying model on CORONA satellite imagery from the 1960s for the Abu Ghraib district west of Baghdad, reporting a segmentation IoU above 85% and confirming four previously unrecorded sites through field verification. Two other groups apply CNNs to CORONA imagery in different regions: Soroush et al. [31] detect qanat shafts in the Kurdistan Region of Iraq, and Vadineanu et al. [36] use explainability techniques on CORONA-trained classifiers to delineate mounded settlements in Upper Mesopotamia. Both confirm that CORONA-based detection is an established, independently replicated task rather than a peculiarity of this dataset, though neither paper reports the per-file storage cost of the imagery, which is this thesis’s specific concern with it.

Two themes from this literature bear directly on the present work. First, Fiorucci et al. [15] argue that standard detection measures are poorly matched to archaeological objects on LiDAR and propose alternatives; Casini et al. [6] make the related observation that IoU and site-level detection accuracy inversely related, because a more conservative model produces fewer positive predictions, shrinking the union term and raising IoU while missing sites and lowering recall. Any model comparison that relies on IoU alone must be read with that caveat.

Second, labels are noisy in a structured way. A substantial fraction of catalogued sites are no longer visible on modern imagery, having been levelled for agriculture or obscured by development; a model that correctly produces no contour for such a tile is penalised as a false negative against the annotation. Casini et al. [6] address this with human-in-the-loop adjudication, which raises reported detection accuracy from roughly 63% to 80%. This is a property of the dataset, not of any particular model, and it bounds what any reproduction can be expected to achieve.

2.3 Input pipelines and I/O for large-scale training

The literature above is concerned with what the model computes. This section looks into the literature discussing gpu utilisation and optimisation techniques. General surveys of GPU-oriented deep-learning optimisation [21] cover compute- and memory-level techniques - kernel fusion, mixed precision, distributed parallelism - but treat the input pipeline as out of scope; the work reviewed below fills that gap specifically.

Mohan et al. [22] characterise *data stalls* in DNN training and show that they are common rather than exceptional: across a range of workloads, GPUs spend a substantial fraction of wall-clock time blocked on fetching and preprocessing, and the effect worsens as GPUs get faster relative to storage and host CPUs. Their system, CoordDL, addresses this through coordinated caching and by eliminating redundant preprocessing across concurrent jobs. Murray et al. [23] describe `tf.data` and make the same argument from the framework side, arguing that input pipelines deserve to be treated as first-class, optimisable programs rather than as glue code. Svogor et al. [32] profile the PyTorch [26] `DataLoader` specifically against high-latency, S3-like object storage and report that a modified `ConcurrentDataLoader` recovers up to a 12× reduction in batch-loading time - independent confirmation, on different storage and a different framework layer, that the many-small-file problem this thesis targets on GPFS is not an artefact of one filesystem.

The many-small-file problem is a specific and well-known bottleneck on parallel filesystems. Metadata cost per file is close to constant regardless of how few bytes are read, so per-file overhead does not scale well, and a dataset of thousands of small images generates metadata traffic out of proportion to the data actually transferred. The standard mitigation is to aggregate samples into large sequentially read archives. WebDataset [1] formalises this: samples are stored in POSIX tar archives and streamed sequentially, converting many small random reads into few large sequential ones, at the cost of exact random-access shuffling - which is approximated by shuffling shard order and maintaining an in-memory sample buffer. NVIDIA DALI [24] attacks an adjacent part of the problem by moving decode and augmentation onto the GPU. The term *IO-aware*, used throughout this thesis for PADS-Loader’s central mechanism, echoes an argument made one level lower in the compute stack, Dao et al. [12] show that attention kernels which account explicitly for the cost of moving data between GPU high-bandwidth memory and on-chip SRAM outperform kernels that minimise floating-point operations alone. The claim in this thesis is the same principle applied one layer higher in the storage hierarchy - between a shared

parallel filesystem and node-local scratch, rather than between High Bandwidth Memory and SRAM.

Two gaps in this body of work motivate PADS-Loader.

First, these systems are *statically configured*. WebDataset streams shards at a fixed depth; the number of loader workers and the prefetch factor are chosen once. None of them observes whether the configuration is in fact sufficient to hide the transfer, and none revises it when contention on a shared filesystem changes the answer mid-run.

Second, and more specifically, they do not exploit the fact that in supervised training with a fixed epoch permutation, *future accesses are known exactly*. Caching and prefetching are ordinarily speculative because future references must be predicted. Here they need not be: once the permutation is drawn, the identity of the next k samples is determined, and a staging schedule can be computed rather than guessed. PADS-Loader is built on that observation.

Segmentation architecture is not treated as solved in general. For this specific workload, however, the reference study found that architectural choice moves IoU by only a few percentage points, plausibly within the fluctuation induced by random augmentation [6]. If the remaining headroom is not in the model, it may be in the pipeline that feeds it. The I/O literature establishes that data stalls are real and that aggregation into shards mitigates the dominant filesystem pathology, but leaves the configuration static and the prefetch speculative. This thesis addresses that gap.

3 Materials and Datasets

3.1 Overview

This chapter describes the data and the model architectures used throughout the thesis. The experimental subject is a semantic segmentation workload over satellite imagery of the southern Mesopotamian floodplain, and the object of study is the data pipeline that feeds that workload on a shared HPC filesystem. The dataset is therefore treated in two distinct ways: as a machine learning dataset, characterised by its imagery, class balance and evaluation metrics (Section 3.2); and as an I/O workload, characterised by file count, file size distribution and access pattern (Section 3.3). The second characterisation is the one that motivates PADS-Loader, and it is the one usually left implicit in the segmentation literature.

3.2 Dataset: Casini et al. (2023)

The main experiments use the dataset introduced by Casini et al. [6], which addresses semantic segmentation and detection of mounded archaeological sites - tells - across the southern Mesopotamian floodplain in Iraq.

3.2.1 Provenance and construction

The dataset originates from a corpus of geo-referenced vector shapes corresponding to the contours of known mound sites within the survey area of the Floodplains Project, spanning approximately 66,000 km². The vector corpus was assembled at the University of Bologna by consolidating all published archaeological surveys in the region and geo-referencing the sites catalogued therein; it contains 4,934 shapes, each corresponding to a site confirmed by ground-truthing and by study of the associated surface scatter of artefacts.

Because the corpus was compiled as an archaeological reference rather than as a training set, the authors applied two filtering passes before image generation:

1. **Removal of the 200 largest sites by area.** These exceed the area of the square input region and would therefore yield an almost entirely positive segmentation mask, providing no usable learning signal. Visual inspection confirmed that they follow the shape of extended areas rather than of simple mounds.
2. **Removal of 684 sites** that were either too small to be a tell or marked as destroyed. The size threshold was set at roughly 1,000 m², corresponding to a circle of 30 m diameter.

Sites below this threshold typically correspond to generic annotations for known sites of unknown size or imprecise location.

Image generation was performed in QGIS [28]. For each retained site, a Python script exported a square of side L centred on the site centroid containing only Bing Maps satellite imagery, accessed through the QuickMapService plugin. The same square was exported a second time without the base map but with the site contour rendered as a solid filled shape, giving the corresponding binary ground-truth mask. The task is therefore binary: one class for mounded sites, one for everything else.

Two spatial configurations were produced: $L = 1000$ m (“1k”) and $L = 2000$ m (“2k”). Imagery was exported at approximately 1 pixel per metre - 1024 px for 1000 m, and double that for the larger extent - and subsequently downsampled by a factor of two to reduce computational cost, which the original authors report as having low impact on accuracy. At training time a square of side $L/2$ is randomly cropped from the exported tile; this both prevents the model from learning a positional prior in which sites always appear at the image centre, and acts as data augmentation. Further augmentation consists of random rotation, mirroring, and small shifts in brightness and contrast, resampled at each training iteration.

Finally, 1,155 images with empty masks - containing no site to predict - were added, sampled from locations proposed by the archaeologists: highly urbanised areas, intensive agriculture, flood-prone terrain such as artificial lakes and basins, and rocky hills and mountains. These negatives are what force the model to learn what a tell is *not*. The final dataset consists of 5,025 image/mask pairs, split 90% training / 10% held-out test, stratified over the empty images, with a further 10% of the training partition randomly reserved as a validation set.

A CORONA-augmented variant also exists, in which declassified historical satellite imagery is imported into QGIS and cropped identically to the Bing tiles, giving a second input channel group. CORONA was a United States photographic-reconnaissance programme whose imagery was declassified in 1995; the USGS archive covers acquisitions from 1960–1972 and notes that the scanned products are not supplied with geocorrection [35]. Because CORONA was used only as a complement in the original work, sites destroyed after the 1970s were excluded from that analysis.

Dataset variant used in this thesis. All experiments reported here use the Bing 1k configuration unless otherwise stated. This choice is deliberate: the 1k configuration is the smallest of the published variants and therefore the least favourable to a data-staging argument - a pipeline that is I/O-bound on 1k tiles will be at least as I/O-bound on 2k tiles or on the CORONA-augmented input. Section 6 revisits the larger input configurations.

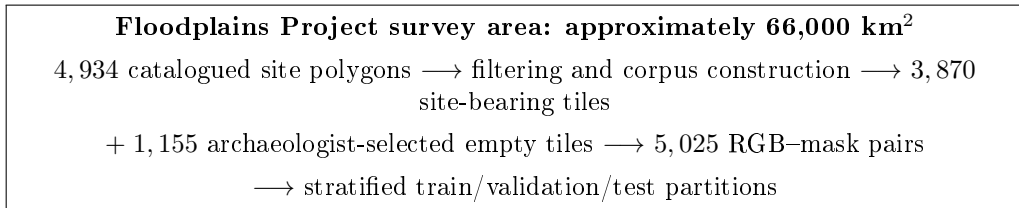


Figure 2: Scope and construction of the dataset used in this thesis. Counts and filtering rules are reported by Casini et al. [6]; the diagram distinguishes the regional catalogue from the image-mask pairs actually presented to the models.

3.2.2 Class imbalance

The segmentation target is severely imbalanced: mound pixels constitute a small minority of any given tile, and 1,155 of the 5,025 tiles contain no positive pixels at all. Direct measurement over the corpus used here gives a mean positive-pixel fraction of approximately 9.6% per site-bearing tile, with a median near 7%. The masks themselves are cleanly binary - 90.4% of mask pixels take the background value and 8.8% the foreground value, with only 0.7% at intermediate values - so label ambiguity at object boundaries is not a material source of error.

Property	Value used in the main experiments
Geographic scope	Southern Mesopotamian floodplain; source survey area approximately 66,000 km ² [6]
Task	Binary semantic segmentation of tell pixels
Samples	5,025 paired RGB images and masks: 3,870 site-bearing and 1,155 empty
Spatial configuration	1,000 m square exported at 1,024 px, then downsampled/cropped
Partitioning	90% train / 10% held-out test; 10% of training reserved for validation
Primary input	Bing satellite imagery; CORONA and 2 km variants excluded from main runs
Storage layout	5,025 image files plus 5,025 mask files on shared GPFS

Table 1: Key properties of the dataset and the Bing 1k variant used throughout the primary evaluation [6].

This imbalance is the reason the original work adopts focal loss [20] over Dice loss, and the reason the primary reported metric is IoU rather than pixel accuracy. Focal loss is one of several loss functions purpose-built for class imbalance in segmentation; Azad et al. [2] survey twenty-five such loss functions and describe focal loss’s reweighting term as reducing the loss contribution of easily-classified samples relative to hard-to-classify ones. In a tile that is 90% background, as here, the great majority of easily-classified pixels are background, so this reweighting acts as an implicit foreground up-weighting – though the survey states the mechanism in general easy/hard terms, not specifically in terms of this dataset’s class split. It is also, importantly for this thesis, the reason the workload is decode-heavy relative to its useful signal: a substantial fraction of the bytes read from the filesystem contribute nothing but negative pixels.

For a pixel with target $y \in \{0, 1\}$ and predicted foreground probability p , binary cross-entropy is

$$\mathcal{L}_{\text{BCE}} = -[y \log p + (1 - y) \log(1 - p)]. \quad (2)$$

Writing $p_t = p$ when $y = 1$ and $p_t = 1 - p$ otherwise, focal loss applies a modulating factor to easy examples [20]:

$$\mathcal{L}_{\text{focal}} = -\alpha_t(1 - p_t)^\gamma \log p_t. \quad (3)$$

The overlap-based alternatives used in the configuration search are Dice and Jaccard losses. For soft prediction vector $\mathbf{p}$ and binary target $\mathbf{y}$, with $\varepsilon > 0$ for numerical stability, they are

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i p_i y_i + \varepsilon}{\sum_i p_i + \sum_i y_i + \varepsilon}, \quad (4)$$

$$\mathcal{L}_{\text{Jaccard}} = 1 - \frac{\sum_i p_i y_i + \varepsilon}{\sum_i p_i + \sum_i y_i - \sum_i p_i y_i + \varepsilon}. \quad (5)$$

3.2.3 Evaluation metrics

The original paper evaluates using intersection-over-union at the pixel level, and accuracy, precision, recall and the Matthews correlation coefficient at the site-detection level:

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} \quad (6)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP} \quad (8)$$

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (9)$$

where P denotes the predicted shape and G the ground-truth shape.

The distinction between the two levels matters. IoU measures how well a predicted contour overlaps an annotated one; it does not measure how many sites were found. The original authors note that IoU can improve while detection accuracy degrades, as occurs between their Model 5 and Model 6: a more conservative model produces fewer positive predictions, shrinking the union term and raising IoU, while missing sites and lowering recall.

Reduction convention. A subtlety that materially affects reported numbers is how per-image scores are aggregated. The reference implementation reports `smp.metrics.iou_score(..., reduction="macro-imagewise")`: IoU is computed per image and then averaged, with the library’s convention that an empty ground truth correctly predicted empty scores 1.0. This differs from a globally pooled IoU, in which true positives, false positives and false negatives are summed across the whole set before the ratio is taken. On the same trained model the two conventions differ by several percentage points, with the per-image reduction the higher of the two because the empty tiles contribute perfect scores. All comparisons in this thesis therefore state which reduction is used. Section A.5 quantifies the difference.

3.2.4 Reference results

Casini et al. report the IoU scores in Table 2 on the held-out test set. Standard deviations arise from repeated testing to average out the random crop, which is applied at test time as well as during training.

Model	Input	Filter	IoU (%)	Std. dev
Model 1	Bing 1k		74.17	0.38
Model 2	Bing 1k	✓	78.10	0.54
Model 3	Bing + CORONA 1k		74.06	0.39
Model 4	Bing 2k		79.77	0.34
Model 5	Bing 2k	✓	81.54	0.35
Model 6	Bing + CORONA 2k	✓	83.45	0.18

Table 2: IoU scores reported by Casini et al. [6] for their experimental configurations.

At the detection level, Model 5 achieves 62.57% accuracy under automatic evaluation and 80.40% after human-in-the-loop adjustment for mislabelled (no-longer-visible) sites; Model 6 achieves 60.08% and 79.13% respectively. Their configuration search compared U-Net against MA-Net, ResNet-18 against EfficientNet-B3, and Dice loss against focal loss, settling on MA-Net + EfficientNet-B3 + focal loss trained for 20 epochs as the best configuration; they observe that the differences between configurations were small, within a few percentage points, and plausibly attributable to fluctuation from the random augmentation.

These figures serve two purposes here. First, they establish an accuracy floor: any optimisation to the data pipeline proposed in this thesis must be shown not to degrade segmentation quality relative to this reference. Second, the observation that architectural choice moves IoU by only a few points motivates the framing of this work - if the model is not where the remaining headroom is, the pipeline may be.

3.3 The dataset as an I/O workload

The above is the standard description of the dataset. For the purposes of this thesis a second description is required, because the property of the dataset that PADS-Loader responds to is not its class balance but its file layout.

The Bing 1k dataset consists of 5,025 image files and 5,025 corresponding mask files stored as individual objects on the shared GPFS filesystem. Each training step therefore issues, per sample, an independent `open/stat/read/close` sequence against a parallel filesystem that is optimised for large sequential transfers rather than for many-small-file metadata operations. The relevant properties are:

Many small files. Metadata cost per sample is close to constant regardless of how few bytes are actually read, so per-file overhead does not amortise.

Paired access. Every image read is accompanied by a mask read at a different path, doubling the metadata traffic per sample.

Shared contention. GPFS is a shared resource; observed read latency is not a property of the dataset alone but of concurrent jobs on the cluster, and is therefore non-stationary across a run.

Deterministic access order. Once the epoch permutation is fixed, the sequence of samples - and hence of files - is known in advance. Future data access is predictable, which is precisely the precondition that makes proactive prefetching and shard staging exact rather than speculative.

The final point is the hinge of the entire method. The dataset is small enough to fit on node-local scratch and its access pattern is known ahead of time; the fact that the baseline pipeline nonetheless leaves the GPU idle is a property of the loader, not of the data.

3.4 Models

Three segmentation architectures are used, spanning a CNN encoder-decoder, an attention-augmented CNN, and a transformer. All three were trained under an identical data pipeline so that pipeline effects can be isolated from architectural effects.

A convolutional neural network (CNN) learns local filters whose receptive field grows with depth; an encoder reduces spatial resolution to obtain contextual features, while a decoder up-samples those features and combines them with fine-resolution information. A vision transformer instead represents the image as a sequence or hierarchy of patches and uses self-attention to relate content across spatial positions [39, 42]. The three models instantiate these principles differently (Figure 3).

MA-Net: EfficientNet-B3 encoder → multi-scale feature maps → position/block attention → U-Net-style decoder.
DeepLabV3+: ResNet-50 encoder → atrous spatial pyramid pooling at multiple dilation rates → boundary-refining decoder.
SegFormer: hierarchical overlapping patch encoder → multi-scale transformer features without positional encoding → lightweight all-MLP decoder.

Figure 3: Architectural paths of the three evaluated models, summarised from the primary architecture papers [14, 10, 42].

3.4.1 MA-Net

MA-Net (Multi-scale Attention Network) [14] extends the U-Net encoder-decoder with a self-attention mechanism inserted in the latent space, allowing the network to weight latent features according to content. Originally developed for liver and tumour segmentation in medical imaging, it has since been applied to remote sensing. It is included here because it is the architecture selected as best-performing by Casini et al., and therefore constitutes the reproduction baseline for this thesis. Implementation: `segmentation_models_pytorch` [18] with an EfficientNet-B3 [33] encoder pretrained on ImageNet [13].

3.4.2 DeepLabV3+

DeepLabV3+ [10] uses atrous (dilated) convolution and atrous spatial pyramid pooling to capture multi-scale context without loss of spatial resolution, combined with a decoder module that recovers object boundaries. It is included as a widely used, high-capacity CNN baseline and represents the heaviest of the three in both parameter count and per-step GPU time. Implementation: `segmentation_models_pytorch` with a ResNet-50 encoder.

3.4.3 SegFormer

SegFormer [42] pairs a hierarchical, positional-encoding-free transformer encoder with a lightweight all-MLP decoder. It is included as the transformer baseline and, critically for this thesis, as the least GPU-bound of the three architectures: its low per-step compute makes it the configuration in which data-loading stalls are most exposed. Implementation: the NVlabs reference architecture as distributed through the HuggingFace `transformers` library.

3.4.4 Measured characteristics (Bing 1k, default loader)

Measurements were collected on the Iris cluster with one V100 and the baseline PyTorch `DataLoader` at default settings (Table 3), and with loader parameters tuned (Table 4) (`num_workers=4`, `pin_memory=True`). The same underlying runs are reported with the RQ1 comparison (Table 9).

	SegFormer	MA-Net	DeepLabV3+
Trainable parameters	3.7 M	17.2 M	26.7 M
Model size (MB)	15	68	106
Batch size	32	32	32
GPU utilisation (%)	16.0	34.4	33.6

Table 3: Baseline characteristics with the default PyTorch `DataLoader` (`num_workers=0`).

	SegFormer	MA-Net	DeepLabV3+
Trainable parameters	3.7 M	17.2 M	26.7 M
Model size (MB)	15	68	106
Batch size	32	32	32
GPU utilisation (%)	40.5	85.1	82.6

Table 4: Characteristics with loader parameters tuned (`num_workers=4`, `pin_memory=True`).

The observation from these tables that sets up Section 4 is that GPU utilisation moves by 24–51 percentage points between them depending on architecture – large enough, together with the per-stage timing breakdown of Section 5.1, that the data path is a plausible bottleneck rather than a rounding error. Loader configuration is where this workload’s throughput headroom demonstrably sits, and that headroom is worth pursuing on its own terms. Segmentation quality is treated throughout as a constraint to be preserved rather than an objective to be advanced; Appendix A reports it directly and confirms that no data-path configuration in this thesis trades accuracy for the throughput gains reported here.

3.5 Hardware and software environment

All primary experiments were run on the University of Luxembourg’s Iris cluster, whose facility architecture is documented by Varrette et al. [38], on GPU nodes equipped with NVIDIA V100 GPUs and scheduled via Slurm. Storage is provided by a shared GPFS filesystem, with node-local scratch available for staging. The software stack is containerised with Apptainer to guarantee a reproducible environment across nodes; the container image bundles PyTorch [26], PyTorch Lightning, `segmentation_models_pytorch` [18] and `albumentations` [5].

A secondary, single-GPU workstation environment (NVIDIA RTX 3050 Laptop, 4 GB) was used for development, defect analysis and reproduction work. Appendix A documents that environment and, importantly, the respects in which results obtained on it are *not* comparable to the cluster results reported above – most consequentially the GPU memory limit, which forbids the batch size of 32 used throughout the primary experiments and thereby degrades batch-normalisation statistics. That constraint is the reason all accuracy figures in this thesis are drawn from the cluster.

4 Methodology

4.1 Motivation: when the DataLoader becomes the bottleneck

Deep learning training on HPC is conventionally treated as a GPU problem. The profiling results reported in Section 3.4.4, together with a targeted profiling pass over the baseline pipeline, indicate that for this workload the GPU is frequently not the constraint. Four findings motivate PADS-Loader:

1. **GPU starvation.** With `num_workers=0`, the GPU sits idle between batches. It is waiting on the data path, not on the model.
2. **Shared filesystem pressure.** Thousands of small image and mask files generate high metadata overhead on GPFS. The cost is dominated not by bytes transferred but by per-file operations against a shared parallel filesystem.
3. **Static configuration.** `num_workers`, `prefetch_factor` and `pin_memory` are set once at loader construction and never adapt. They cannot respond to a bottleneck that shifts - as it does, when contention on a shared filesystem varies over the lifetime of a job.
4. **Deterministic access.** Because the sample order is fixed once the epoch permutation is drawn, the sequence of future file accesses is known. Prefetching is therefore not speculative: it can be exact.

4.2 PADS-Loader: system overview

PADS-Loader (Profiler-Adaptive Data Staging) is a drop-in replacement for the PyTorch `DataLoader` that observes its own behaviour during a short warm-up window, classifies the dominant bottleneck from the observed timings, selects a staging policy accordingly, and then executes that policy with predictive prefetching for the remainder of the run.

The pipeline has four stages:

1. **Warm-up profiling** - 50–100 batches, instrumented.
2. **Bottleneck classification** - I/O, decode, H2D or compute.
3. **Policy selection** - loose, shard or stage.
4. **Predictive staging** - shard prefetch to node-local scratch.

The design intent is that stages 1–3 cost a bounded, one-off overhead - tens of seconds on a job measured in hours - and that stage 4 recovers that cost many times over. The loader is deliberately conservative by default: if no bottleneck is detected, it degrades to the loose policy, which is the ordinary PyTorch `DataLoader`.

4.2.1 Stage 1 - Warm-up profiling

For the first 50–100 batches, PADS-Loader instruments the data path and records per-batch timings without altering behaviour. The signals measured are given in Table 5.

The final two are feasibility signals rather than performance signals. They determine whether a policy is *available*, not whether it is desirable.

Instrumentation. The four timers are implemented as follows. T_{gpu} and T_{h2d} are measured with `torch.cuda.Event` pairs recorded around the transfer and the forward/backward call respectively, synchronised before reading, because wall-clock timing of asynchronous CUDA work would otherwise attribute queueing delay to the wrong stage. T_{data} is measured as wall-clock time around `next(iterator)`, which captures the time the training loop is blocked waiting for a batch - and is therefore the signal that directly expresses starvation. T_{decode} is instrumented inside `__getitem__`, accumulating per-sample decode and transform time, and is aggregated per batch.

Signal	Meaning
T_{data}	Time spent waiting for the next batch to become available
T_{decode}	Image and mask decode time, plus augmentation transforms
T_{h2d}	Host-to-device (CPU to GPU) transfer time
T_{gpu}	Model forward + backward time
GPU utilisation	GPU occupancy over the window
Queue fullness	<code>DataLoader</code> output-queue depth
Shard size	Whether the next shard can be staged efficiently
Scratch availability	Whether node-local scratch has free capacity

Table 5: Signals recorded during the warm-up window.

The output queue is sampled periodically to measure how full it is. If the queue remains consistently full, the `DataLoader` is producing batches faster than the model can consume them, indicating that model computation is the bottleneck. Conversely, if the queue remains consistently empty, the model is consuming batches faster than the `DataLoader` can produce them, indicating that data loading is the bottleneck. It therefore distinguishes between a genuinely efficient data-loading pipeline and a low T_{data} that occurs simply because a slow model gives the `DataLoader` enough time to prefetch and prepare the next batch.

Aggregation. Statistics are taken as the median over the window rather than the mean. The first batches of a run include CUDA context creation, cuDNN algorithm selection and worker process start-up, all of which are one-off costs that would inflate a mean and bias the classifier towards diagnosing an I/O bottleneck that does not exist in steady state. The first $w_0 = 10$ batches are discarded before statistics are computed, and the median is taken over the remainder.

4.2.2 Stage 2 - Bottleneck classification

The profiled signals are mapped onto a dominant bottleneck class. The classification is not a black box: it is a small set of rules over the measured timings, chosen so that the decision is inspectable and reproducible.

Detected condition	Interpretation	Action
T_{data} low, T_{gpu} high, GPU utilisation high	Compute-bound; the pipeline is already keeping up	Loose
Many <code>open/stat/read</code> calls; high metadata overhead	Many-small-file overhead dominates	Shard
Shared filesystem read latency high; scratch available	Shared-FS latency dominates and can be avoided	Stage
Many-small-file overhead <i>and</i> slow shared FS	Both bottlenecks present	Shard + Stage
T_{decode} dominates	CPU-side decode is the constraint	Increase workers / cache decoded samples

Table 6: Bottleneck classification rules.

Policies are not mutually exclusive. Scratch staging and an increased worker count may both be active simultaneously when filesystem I/O and decode are co-bottlenecks. The classifier therefore emits a *set* of actions rather than a single label.

Threshold-based policy selection. The policy-selection rules use relative timing ratios rather than fixed time thresholds, allowing them to remain applicable across GPUs and systems with dif-

ferent performance characteristics. Writing $\tau = \max(T_{\text{decode}}, T_{\text{gpu}})$, where τ represents the larger of data-decoding time and GPU computation time, the implementation used here declares:

$$\text{stage} \quad \text{if} \quad \text{scratch available} \wedge T_{\text{h2d}} > \gamma_s \cdot \max(\tau, \epsilon) \quad (10)$$

$$\text{shard} \quad \text{if} \quad T_{\text{decode}} > \gamma_d \cdot \max(T_{\text{h2d}}, T_{\text{gpu}}) \vee T_{\text{data}} > \gamma_d \cdot \tau \quad (11)$$

$$\text{loose} \quad \text{otherwise} \quad (12)$$

The **stage** policy is selected when node-local scratch storage is available and the host-to-device transfer time, T_{h2d} , exceeds the dominant processing time τ by a factor of γ_s . The **shard** policy is selected when either the decoding time, T_{decode} , significantly exceeds both host-to-device transfer and GPU computation time, or when the observed data-waiting time, T_{data} , is large relative to τ . If neither condition is satisfied, the loader retains the default **loose** policy.

A note on what the classifier optimises. The rules above ask “is the data path the bottleneck?”, not “which reader is fastest?”. These are different questions, and the distinction matters when interpreting results. If T_{data} is fully hidden behind T_{gpu} by worker overlap, a reader that is intrinsically faster buys no wall-clock improvement, and the classifier correctly declines to switch. Section 6.3 examines the sensitivity of the outcome to γ_s and γ_d .

4.2.3 Stage 3 - Policy selection

Three staging policies are defined, summarised in Table 7.

Policy	What it does	Best when
Loose	Reads individual image and mask files directly from GPFS	Simple baseline; small or node-local dataset; workload already compute-bound
Shard	Reads samples from tar archives, amortising per-file metadata cost over many samples	Many-small-file overhead is the problem
Stage	Copies future shards to node-local scratch ahead of use, removing the shared filesystem from the critical path	Shared filesystem latency is the problem

Table 7: The three staging policies.

Loose is the baseline and the fallback. Shard attacks metadata overhead by converting thousands of **open** calls into a handful of large sequential reads. Stage attacks latency and contention by moving data off the shared filesystem entirely, exploiting the fact that the dataset fits in node-local scratch. The two compose: shards may be both formed and staged.

Archive construction. The shard policy requires archives whose member layout the reader can address by original filename. Archives are built once, offline, as uncompressed POSIX tar files with members named `images/<name>.jpg` and `masks/<name>.png`. Compression is deliberately not applied: the payload is already-compressed JPEG and PNG, so entropy coding yields negligible size reduction while forcing the archive to be decompressed from the stream start on every random access - precisely the access pattern a shuffled loader produces. This is a departure from WebDataset [1], which streams shards sequentially and therefore does not pay that penalty; the choice here preserves exact random access at the cost of forgoing compression.

4.2.4 Stage 4 - Predictive staging

Because the epoch permutation is fixed at the start of each epoch, the order in which the shards will be consumed is known in advance. PADS-Loader uses this information to stage future shards before they are required. While shard i is being consumed by the training process, a background

staging thread copies a future shard $i + d$ from the shared filesystem to node-local scratch, where d denotes the prefetch depth. The objective is to overlap the transfer from shared storage with model computation so that the required shard is already available locally when the training process reaches it.

The prefetch depth d is not a hyperparameter supplied by the user. Instead, it is derived automatically from runtime measurements. Let t_{stage} denote the measured time required to copy one shard from the shared filesystem to node-local scratch, and let t_{consume} denote the measured time required by the training process to consume one shard’s worth of samples. The depth is set to

$$d = \min! \left(\left\lceil \frac{t_{\text{stage}}}{t_{\text{consume}}} \right\rceil + 1, ; d_{\text{max}} \right), \quad d_{\text{max}} = \left\lfloor \frac{C_{\text{scratch}}}{s_{\text{shard}}} \right\rfloor. \quad (13)$$

Here, the ratio $t_{\text{stage}}/t_{\text{consume}}$ estimates the number of shard-consumption intervals required to cover the transfer time of one shard. For example, if staging a shard takes 12 seconds while consuming a shard takes 5 seconds, the transfer spans approximately $12/5 = 2.4$ consumption intervals. Rounding this value upward gives three shards, and the additional $+1$ provides one shard of safety margin, resulting in a prefetch depth of four. This additional shard provides protection against variations in t_{stage} , which may occur on a shared filesystem due to changing system load and I/O contention.

The calculated depth is additionally constrained by the available node-local scratch capacity. Let C_{scratch} denote the free capacity of node-local scratch and s_{shard} the size of a single shard. The resulting value

$$d_{\text{max}} = \left\lfloor \frac{C_{\text{scratch}}}{s_{\text{shard}}} \right\rfloor$$

represents the maximum number of shards that can be stored in scratch at the same time. The final prefetch depth is therefore the smaller of the depth required to hide the measured staging latency and the number of shards that can physically fit in the available scratch space.

This mechanism allows the prefetch depth to adapt automatically to the observed storage and training performance. If t_{stage} increases, shards must be requested further in advance and the required prefetch depth increases. Conversely, if t_{consume} increases, the staging thread has more time to complete a transfer while existing shards are being consumed, reducing the required prefetch depth. The prefetch depth is therefore determined by measured system behaviour rather than by a manually selected value.

This is the mechanism by which RQ4—can prefetch depth be auto-tuned from measured timings?—is answered affirmatively by construction. It also distinguishes PADS-Loader from WebDataset, where shards are streamed using a fixed, statically configured prefetch depth without determining whether that depth is sufficient to hide the measured transfer latency.

Eviction. Once a staged shard has been fully consumed, its local copy is removed from node-local scratch to make space for subsequent shards. Since the shard consumption order is known in advance, PADS-Loader can determine exactly when a shard is no longer required and evict it immediately after consumption. This maintains at most d resident shards and avoids the need for recency-based cache-management heuristics such as least-recently-used (LRU) eviction.

4.3 Metrics captured

Two families of metric are reported.

Pipeline metrics (the object of the optimisation).

- **GPU utilisation** - mean GPU occupancy over the run, the headline indicator of whether starvation has been eliminated.
- **Tile throughput** - samples processed per second; the end-to-end figure of merit.
- **Time-to-epoch** - wall-clock seconds per epoch, and total time-to-target-IoU.

- $T_{\text{data}} / T_{\text{decode}} / T_{\text{h2d}} / T_{\text{gpu}}$ **breakdown** - retained for the full run, not just warm-up, so that the *shift* in the bottleneck under each policy is visible rather than merely its disappearance.
- **Filesystem operation counts** - open/stat/read calls issued against GPFS, the direct measure of whether sharding achieved what it was intended to achieve (RQ2).
- **Scratch traffic and residency** - bytes staged, staging hit rate, peak scratch occupancy.
- **Profiling overhead** - the cost of the warm-up window itself, reported explicitly so that the net benefit is honest.

Model metrics (the constraint, not the objective).

- IoU on the held-out test set, against the Section 3.2 reference, stating the reduction convention used.

The reason for reporting both is that this thesis makes a negative claim about accuracy as well as a positive claim about throughput. PADS-Loader is only useful if it accelerates the pipeline *without* degrading segmentation quality. A throughput improvement obtained at the cost of reduced shuffling randomness, and therefore of a worse model, is not an improvement. Any observed IoU deviation greater than the ± 0.5 percentage point standard deviation reported in the reference results must be treated as a regression to be explained, not as noise.

4.4 Evaluation plan

PADS-Loader is compared against four alternatives, all on identical model, dataset and hardware, with only the data path varying (Table 8).

#	Method	What it isolates
1	Baseline PyTorch <code>DataLoader</code> – loose files, default settings	The unmodified starting point
2	Tuned PyTorch <code>DataLoader</code> – best <code>num_workers</code> , <code>pin_memory</code> , <code>prefetch_factor</code>	How much is recoverable by manual tuning alone; PADS-Loader must beat this, not merely the naive baseline
3	WebDataset – fixed tar-shard streaming, no adaptation	Isolates the benefit of sharding from the benefit of adaptation (RQ2 vs RQ3)
4	Scratch baseline – loose files copied manually to node-local scratch up front	Isolates the benefit of staging from the benefit of predicting what to stage; the ceiling for any policy that removes GPFS from the path
5	PADS-Loader (proposed) – adaptive staging + predictive prefetch	—

Table 8: Evaluation methods and the contrast each provides.

The comparison set is constructed so that each sub-question is answered by a specific contrast rather than by the aggregate result:

- **RQ1** is answered by the profiling breakdown across methods 1 and 2 and across the three architectures - SegFormer, with the lowest T_{gpu} , is the configuration in which the loader bottleneck should appear first.
- **RQ2** is answered by 3 vs 1, and by the filesystem operation counts.
- **RQ3** is answered by 5 vs 3, with 4 as the upper bound.

- **RQ4** is answered by 5 vs 3, and by a sweep of fixed depths in the ablation of Section 6 - the auto-tuned depth should land at or near the best fixed depth, without having been told it.

Method 4 deserves emphasis: it is the honest ceiling. If manually pre-staging the entire dataset to scratch matches PADS-Loader, then adaptivity has bought nothing for this dataset, and the contribution reduces to the observation that the dataset fits in scratch. PADS-Loader’s claim is that it reaches that ceiling without requiring the user to know in advance that scratch is the answer, and that it continues to work when the dataset does *not* fit - which is the regime the design is actually aimed at, and which Section 6 probes by synthetically constraining scratch capacity.

4.5 Experimental protocol

To ensure that pipeline differences are not confounded with training noise, all comparisons hold the following fixed: random seed, epoch permutation, model architecture and initialisation, batch size, optimiser and learning rate, and the augmentation pipeline. Each configuration is intended to be repeated multiple times and reported as mean $\pm$ standard deviation, because measurements on a shared filesystem are non-stationary and a single run is not strong evidence on its own. This was not achieved for the RQ1–RQ4 and Section 6 evaluation sweep as actually run: cluster time constraints meant each of the 91 configurations across that sweep was run once ($n = 1$), and the GPU-utilisation and IoU figures reported in Sections 5 and 6 should be read as single-run point estimates, not as means with an established variance. This is a real limitation of the evidence collected, not merely of its reporting, and is stated plainly rather than implied by omitted error bars (Section 8 returns to it). The one part of this thesis’s evaluation that does use repeats is the accuracy-reproduction result of Appendix A, where each architecture’s IoU is a mean over ten independent test-set passes with a reported range, following the reference study’s own protocol; that repetition is over test-time crop sampling for a fixed trained model, not over independent training runs, and does not substitute for the repeats this section originally specified. Jobs are submitted through Slurm with identical resource requests, and where possible scheduled on the same node class to control for hardware variation, which remains the intended protocol for any repeated version of this evaluation. This chapter defines PADS-Loader, its measured signals, decision rules, staging policies, and the protocol used to evaluate them. It follows the workload characterisation in Section 3 and supplies the mechanisms tested in Sections 5 and 6.

5 Results

This chapter reports the evaluation described in Section 4.4, across all three architectures – DeepLabV3+, MA-Net and SegFormer – unless stated otherwise. Section 5.1 establishes when the loader is the bottleneck (RQ1); Sections 5.2–5.4 address sharding, predictive staging and prefetch-depth auto-tuning in turn. Section 5.7 verifies the accuracy constraint, and Section 5.6 reports a measurement defect discovered while interpreting the RQ4 and Section 6 results, since it changes how several of the numbers below should be read.

5.1 RQ1: When does the DataLoader become the bottleneck?

The prediction under test was directional and stated in advance: architectures with lower per-step compute time should expose the DataLoader bottleneck more strongly because less GPU computation is available to overlap with data loading. In other words, for a fixed data-loading delay, a model with a short forward and backward pass spends a larger proportion of each training step waiting for data than a model with a longer computation time. SegFormer was therefore expected to be the configuration in which the loader bottleneck appeared most clearly, since its measured per-step GPU compute time of 0.031 s is approximately half that of MA-Net (0.062 s) and DeepLabV3+ (0.065 s).

The results in Table 9 are consistent with this prediction. SegFormer records the lowest tuned GPU utilisation at 40.5%, indicating that its comparatively short computation time leaves

Architecture	Loader	GPU util. (%)
SegFormer	default (<code>num_workers=0</code>)	16.0
SegFormer	tuned (<code>num_workers=4</code>)	40.5
MA-Net	default	34.4
MA-Net	tuned	85.1
DeepLabV3+	default	33.6
DeepLabV3+	tuned	82.6

Table 9: GPU utilisation, default vs. tuned DataLoader, all three architectures. Two loader flags – raising `num_workers` from 0 to 4 – move utilisation by 24–51 percentage points depending on architecture.

less opportunity to hide data-loading latency behind GPU execution. In contrast, MA-Net and DeepLabV3+ have substantially longer and nearly identical per-step compute times, differing by less than 5%. Their similar default GPU utilisations of 34.4% and 33.6%, respectively, are therefore not evidence against the prediction; rather, the hypothesis provides little reason to expect a meaningful ordering between two architectures whose measured compute times are so close.

The relevant separation is consequently between SegFormer and the two heavier-compute architectures, rather than between MA-Net and DeepLabV3+. SegFormer’s per-step compute time is approximately half that of either CNN architecture, making the same data-loading delay proportionally more significant within each training step. The observed results therefore support the predicted relationship between per-step compute time and sensitivity to the DataLoader bottleneck: faster model computation exposes data-pipeline latency more readily, whereas longer computation provides a larger window in which data loading can be overlapped with GPU execution.

Architecture	T_{data} (s)	T_{decode} (s)	T_{h2d} (s)	T_{gpu} (s)
DeepLabV3+	0.672	0.021	< 0.001	0.065
MA-Net	0.748	0.023	< 0.001	0.062
SegFormer	0.636	0.020	< 0.001	0.031

Table 10: Per-stage timing breakdown, batch size 32, `num_workers=0`, measured under the corrected profiler methodology of Section 5.6 (five warm-up batches discarded, then averaged over a full epoch). T_{h2d} is sub-millisecond throughout and never the bottleneck in any measurement in this thesis. Table 9’s GPU utilisation figures are the available evidence for that comparison.

T_{data} dominates T_{gpu} by a wide margin under the default configuration across all three architectures – 8–20 $\times$. T_{decode} and T_{h2d} are both negligible throughout: T_{h2d} never exceeds one millisecond in any run in this thesis, confirming it is not a plausible bottleneck for this workload regardless of architecture or policy. This is the load-bearing evidence for the rest of the thesis: training on this corpus, on this filesystem, is I/O-bound rather than compute-bound by a wide margin, which is what motivates a data-staging system as the correct intervention.

5.2 RQ2: Do tar shards reduce many-small-file overhead?

The direct measure is filesystem operation counts, not wall-clock: sharding is intended to convert many small metadata operations into few large sequential reads, and that intent is verifiable independently of whether it happens to pay off in this configuration.

At roughly 64 `open` and 32 `stat` calls per 32-item batch – two filesystem operations per sample, one for the image and one for the mask – every sample under `loose` incurs its own metadata round-trip to the shared filesystem. This is the explanation for RQ1’s finding: T_{data} ’s dominance over T_{gpu} is not primarily a bandwidth problem (5.45 MB/batch is trivial for GPFS) but a metadata-operation-count problem, which is exactly the pathology sharding is designed to amortise by consolidating many small files into few large sequential archives.

Policy	open/batch	stat/batch	Bytes/batch
loose (at $\gamma_d = 1.5$)	63.7	31.8	5,450,413

Table 11: Filesystem operations issued against GPFS per batch (batch size 32, `num_workers=0`, consistent across all three architectures since the reader and dataset composition are identical).

5.3 RQ3: Does predictive staging outperform fixed streaming?

Architecture	Method	GPU util. (%)
DeepLabV3+	3. WebDataset	75.4
	4. Full pre-stage (ceiling)	84.2
	5. PADS-Loader	79.5
MA-Net	3. WebDataset	79.1
	4. Full pre-stage (ceiling)	83.3
	5. PADS-Loader	84.8
SegFormer	3. WebDataset	40.1
	4. Full pre-stage (ceiling)	52.8
	5. PADS-Loader	39.5

Table 12: End-to-end comparison, methods 3–5 (methods 1–2 reuse Table 9’s default/tuned runs, which are identical configurations). Segmentation quality under each of these configurations is reported in Appendix A, Table 24.

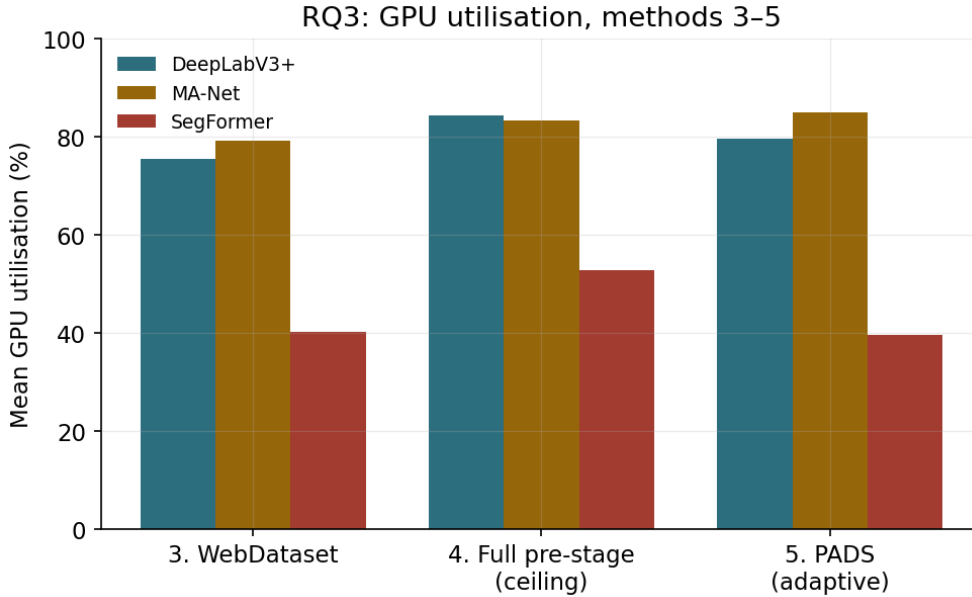


Figure 4: GPU utilisation, RQ3 methods 3–5, all three architectures.

Two comparisons are important for evaluating PADS-Loader, and both show that its adaptive configuration does not outperform the corresponding alternatives consistently across the three architectures.

First, PADS-Loader is compared against method 4, which represents the full pre-staging ceiling by placing the required data on node-local scratch before training begins. PADS-Loader matches or slightly exceeds this ceiling only for MA-Net, reaching 84.8

Second, PADS-Loader is compared against method 2, the tuned baseline reported in Table 9.

Here, PADS-Loader again provides no decisive improvement. For DeepLabV3+, PADS-Loader reaches 79.5

The reason for this result is established in Section 5.6. The adaptive profiler selected `loose` for all three method-5 runs reported in this table. Consequently, PADS-Loader did not activate its staging mechanism in any of these experiments. In practice, these runs therefore behave similarly to the tuned baseline: samples are read as individual loose files using a worker configuration comparable to the tuned baseline.

This result should therefore be interpreted as a limitation of the adaptive decision layer rather than evidence that staging itself is ineffective. Method 4 provides an empirical ceiling for the benefit of staging and shows that moving the dataset to node-local scratch can improve GPU utilisation, with the largest improvement occurring for SegFormer. This result is particularly relevant because SegFormer has the shortest measured per-step compute time and is therefore the architecture for which data-loading latency is expected to be most exposed. The underlying staging mechanism can thus provide a measurable benefit, but PADS-Loader fails to capture that benefit in these runs because its profiler does not select the staging policy. Section 5.6 attributes this behaviour to a specific and quantified measurement bias in the profiler, rather than treating the observed shortfall as an unexplained failure of the staging mechanism itself.

5.4 RQ4: Can prefetch depth be auto-tuned?

Architecture	GPU util. (%) at fixed depth d			
	$d = 1$	$d = 2$	$d = 4$	$d = 8$
MA-Net	82.6	82.4	76.9	77.9
DeepLabV3+	50.3	60.9	65.9	47.3
SegFormer	25.1	39.3	48.0	39.4

Table 13: GPU utilisation across the fixed prefetch-depth sweep. Increasing the depth does not produce a monotonic improvement for DeepLabV3+ or SegFormer: both architectures reach their highest utilisation at $d = 4$ before declining at $d = 8$. MA-Net behaves differently, remaining comparatively flat across the sweep and maintaining high GPU utilisation regardless of the selected staging depth.

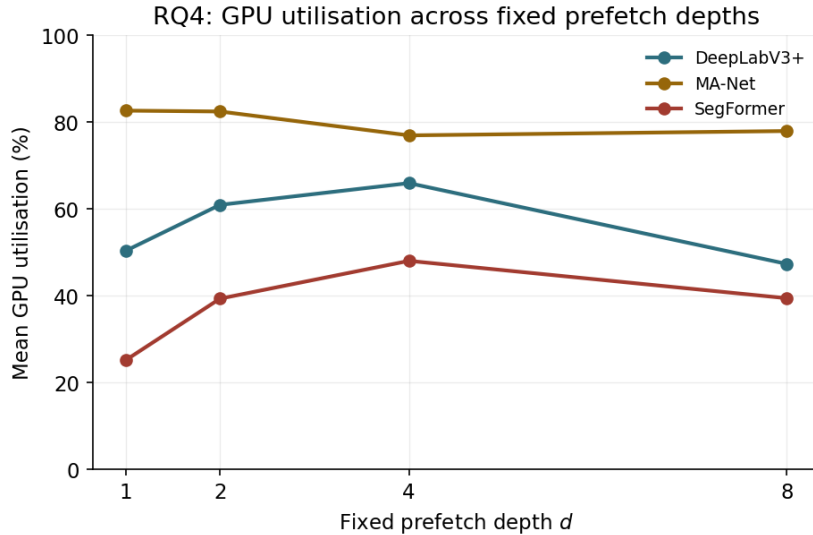


Figure 5: GPU utilisation across the fixed prefetch-depth sweep for all three architectures. DeepLabV3+ and SegFormer reach their highest utilisation at $d = 4$, whereas MA-Net remains comparatively insensitive to the selected depth.

The fixed-depth sweep in Table 13 shows that increasing prefetch depth does not necessarily improve GPU utilisation. DeepLabV3+ increases from 50.3% at $d = 1$ to a maximum of 65.9% at $d = 4$, before falling to 47.3% at $d = 8$. SegFormer follows a similar pattern, increasing from 25.1% at $d = 1$ to 48.0% at $d = 4$, before declining to 39.4% at $d = 8$. MA-Net, by contrast, already reaches 82.6% utilisation at $d = 1$ and remains relatively insensitive to further increases in depth. The results therefore indicate that deeper prefetching is beneficial only up to the point at which the transfer latency is sufficiently hidden; increasing the depth beyond this point does not provide an additional utilisation benefit.

Architecture	T_{stage}	$T_{consume}$	Auto-tuned d
DeepLabV3+	5.47 s / 1.5 ms*	12.8 ms	235 / 2*
SegFormer	5.7 s / 5.69 s	24.5 ms	172 / 234
MA-Net	4.59 s	22.6 ms	205

Table 14: Auto-tuned depth calculated using $d = \min(\lceil T_{stage}/T_{consume} \rceil + 1, d_{max})$. *DeepLabV3+’s two measurements were taken minutes apart on identical hardware, yet differ by approximately $3,600\times$ in T_{stage} alone.

The claim under test is directly falsifiable: the auto-tuned procedure should select a prefetch depth at or near the best-performing fixed depth without being given that value in advance. The results in Table 14 do not support this claim. With the exception of DeepLabV3+’s anomalous second measurement, the automatically selected depths lie between approximately $d = 170$ and $d = 235$. These values are one to two orders of magnitude larger than the $d = 1\text{--}8$ range evaluated in the fixed-depth sweep. More importantly, Table 13 shows that $d = 4$ already provides the highest observed utilisation for DeepLabV3+ and SegFormer. The auto-tuning procedure therefore does not converge towards the region in which the fixed-depth experiments demonstrate that useful values lie.

The source of this discrepancy is the staging-time measurement, T_{stage} , used by the depth-derivation formula in Section 4. The measurements in Table 14 show that T_{stage} is not sufficiently stable under the observed shared-filesystem conditions. Most strikingly, two DeepLabV3+ measurements taken minutes apart on identical hardware report staging times of 5.47 s and 1.5 ms, a difference of approximately $3,600\times$. Since the calculated prefetch depth depends directly on the ratio $T_{stage}/T_{consume}$, this variation propagates directly into d : the two measurements produce auto-tuned depths of 235 and 2, respectively. The problem is therefore not simply that the formula estimates the optimal depth imprecisely; one of its primary inputs can vary by several orders of magnitude between repeated measurements.

The fixed-depth experiments provide independent evidence of the same shared-filesystem variability. SegFormer’s first $d = 8$ run stalled for 66 minutes because of a filesystem delay before completing normally when repeated. This represents the same underlying phenomenon appearing in a different part of the experiment: contention on the shared filesystem can cause an individual staging operation to take substantially longer than its typical duration. During auto-tuning, such an event can distort T_{stage} and therefore the calculated prefetch depth; during training, it instead appears as an abnormally long-running or stalled experiment.

Section 4 already anticipates variability in T_{stage} on a shared filesystem, which is the stated reason for including the additional “+1” shard as a safety margin in the depth formula. However, this mechanism assumes comparatively small fluctuations in staging time and cannot compensate for the three-to-four-order-of-magnitude variation observed here. The automatically derived depths therefore do not fall within, or consistently bracket, the useful region identified by the fixed-depth sweep. This result is reported as a specific and quantified limitation of the current depth-derivation formula: its effectiveness depends on a sufficiently stable estimate of T_{stage} , an assumption that does not hold reliably under the shared-filesystem conditions observed in these experiments.

Warm-up window	Wall-clock (s)
Uncorrected default (8 batches, no discard)	12–15
Full-epoch probe (143 batches, diagnostic only)	107–137
Corrected default (discard 5, measure 8)	$\approx 15^*$

Table 15: Cost of adaptation, measured directly in Section 5.6. * Estimated from 13 total batches processed (5 discarded and 8 measured), compared with 8 batches for the uncorrected default at a comparable per-batch cost.

5.5 Profiling overhead

The profiling procedure described in Section 5.6 introduces only a small increase in profiling cost compared with the uncorrected procedure it replaces. The corrected configuration processes 13 batches in total—5 discarded warm-up batches followed by 8 measured batches—compared with 8 measured batches in the original implementation. In contrast, the full-epoch diagnostic probe processes 143 batches and incurs approximately 8–10 $\times$ greater wall-clock overhead. Although the full-epoch probe provides the most accurate measurement obtained in this thesis, its cost makes it unsuitable as a default profiling procedure. The discard-then-measure approach therefore provides a practical compromise: it retains measurement accuracy close to the full-epoch probe while keeping the profiling overhead close to that of the original short probe.

5.6 Why the adaptive layer stayed conservative: a profiler measurement defect

The results from RQ3 and RQ4 reveal the same behaviour: PADS-Loader’s adaptive profiler almost never selected staging, even in configurations where the fixed-policy experiments demonstrated that staging could improve performance. This section investigates the cause of that behaviour and identifies a measurement defect in the profiler rather than treating the conservative policy selection as an unexplained limitation of the adaptive design.

PADS-Loader’s policy decision is driven by a single warm-up probe at the beginning of each run. By default, this probe measures 8 batches, corresponding to approximately 12–15 s of wall-clock time, in a fixed and unshuffled order. To determine whether this short initial window accurately represents steady-state training behaviour, the same profiling procedure was repeated using `-profile_batches 147`, corresponding to a complete epoch, for all three architectures.

Run	T_{data} (s)	T_{gpu} (s)	Ratio	Policy chosen
DeepLabV3+, 8-batch (default)	0.783	0.613	1.28	loose
DeepLabV3+, 147-batch (full epoch)	0.808	0.100	8.07	shard
MA-Net, 8-batch (default)	0.873	0.713	1.22	loose
MA-Net, 147-batch (full epoch)	0.874	0.093	9.41	shard
SegFormer, 8-batch (default)	0.752	0.655	1.15	loose
SegFormer, 147-batch (full epoch)	0.841	0.062	13.5	shard

Table 16: Comparison between the original 8-batch probe and a full-epoch measurement for the same architecture and configuration. T_{data} remains approximately unchanged, whereas T_{gpu} decreases by 6–10 $\times$ when measured over a complete epoch rather than only the first 8 batches.

Table 16 shows that increasing the profiling window has little effect on T_{data} . The major change occurs in T_{gpu} , which decreases by approximately 6–10 $\times$ when measured over a complete epoch. Consequently, the $T_{\text{data}}/T_{\text{gpu}}$ ratio increases from approximately 1.15–1.28 under the original 8-batch probe to 8.07–13.5 over the full epoch. This change is sufficiently large to alter the resulting policy decision from **loose** to **shard** for all three architectures.

The cause of this distortion is a warm-up effect that was already anticipated in the methodology. Section 4 explicitly identifies the first batches of a training run as potentially unrepresentative because they include one-time costs such as CUDA context creation, cuDNN algorithm

selection, worker-process start-up, and CUDA allocator warm-up. These costs do not represent steady-state training and can therefore inflate the measured GPU time when included in a short profiling window. The methodology consequently specifies two safeguards: discarding the first $w_0 = 10$ batches and calculating the **median** over the remaining measurements rather than their mean.

The shipped implementation of `profile_data_pipeline()`, however, applied neither safeguard. It discarded no initial batches and calculated a plain mean over all 8 measured batches. Because the profiling window was short, the one-time initialisation costs contributed disproportionately to the resulting T_{gpu} estimate. Table 16 makes the resulting bias visible: when these initial costs are diluted across a complete epoch, T_{gpu} approaches its steady-state value while T_{data} remains largely unchanged. The inflated T_{gpu} produced by the original profiler therefore suppresses the $T_{\text{data}}/T_{\text{gpu}}$ ratio, making the pipeline appear substantially more balanced than it is during steady-state training. This explains why nearly every run in Section 6 remained on the conservative `loose` policy.

The full-epoch diagnostic confirms the importance of one of the two safeguards specified in Section 4, but it does not test both. The discard mechanism was subsequently implemented and evaluated below. The specified median aggregation was not implemented or experimentally tested and therefore remains an open issue. Section B.4 of Appendix B records this explicitly rather than including it within the corrected-profiler claim.

Although a 147-batch probe substantially reduces the warm-up bias, it is not a practical default profiling strategy. As shown in Table 15, the full-epoch probe requires approximately 130 s, compared with only 12–15 s for the original 8-batch probe. The relevant question is therefore not simply how large the profiling window should be, but whether a larger measurement window is required at all. An alternative is to exclude the initial warm-up batches and retain a short measurement window once the system has entered a representative steady-state regime.

To test this, `profile_data_pipeline()` was extended with a `discard_batches` parameter, which processes N batches without measurement before beginning the timed profiling loop. A `checkpoint_batches` parameter was also introduced to record the running $T_{\text{data}}/T_{\text{gpu}}$ ratio at several post-discard window sizes within a single run. This allows multiple candidate measurement-window sizes to be evaluated without requiring a separate complete experiment for each window size.

Architecture	8 batches	16 batches	32 batches	64 batches	143 (full)
DeepLabV3+	10.62 (+2.7%)	10.55 (+2.0%)	10.55 (+2.0%)	10.46 (+1.2%)	10.33
SegFormer	20.14 (−0.5%)	20.39 (+0.7%)	20.57 (+1.6%)	20.56 (+1.6%)	20.24
MA-Net	12.18 (+1.6%)	12.20 (+1.8%)	12.23 (+2.0%)	12.22 (+1.9%)	11.99

Table 17: Measured $T_{\text{data}}/T_{\text{gpu}}$ ratio by post-discard window size after discarding the first 5 batches, compared with each architecture’s own 143-batch full-epoch measurement as ground truth. Percentages indicate deviation from the corresponding full-epoch measurement.

The results in Table 17 show that a long measurement window is not required once the initial warm-up batches are excluded. After discarding the first 5 batches, all three architectures produce a $T_{\text{data}}/T_{\text{gpu}}$ ratio within 3% of their respective full-epoch measurements using only 8 additional measured batches. Increasing the measurement window to 16, 32, or 64 batches produces little further improvement. The remaining error also does not decrease monotonically with increasing window size. For example, DeepLabV3+’s 64-batch measurement is closer to its full-epoch value than its 32-batch measurement. This behaviour is consistent with the remaining 1–3% difference being ordinary sampling variation rather than residual warm-up bias.

The defect can therefore be characterised more precisely as a **warm-up contamination problem**, rather than a profiling-window-size problem, consistent with the failure mode already described in Section 4. In these experiments, discarding the first 5 batches and measuring the following 8 requires only 13 batches in total while recovering approximately the same $T_{\text{data}}/T_{\text{gpu}}$ ratio as the 143-batch full-epoch probe. The corrected procedure therefore achieves measurement accuracy close to the full-epoch diagnostic at a fraction of its wall-clock overhead.

This result should not, however, be interpreted as demonstrating that 5 discarded batches

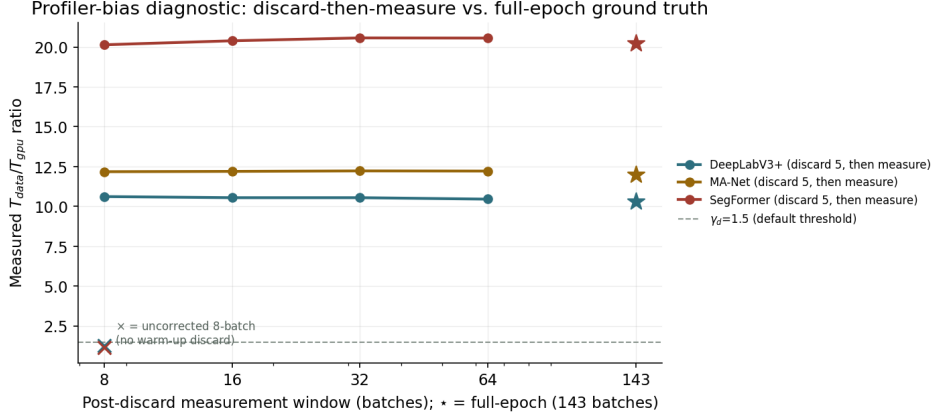


Figure 6: Measured $T_{\text{data}}/T_{\text{gpu}}$ ratio by post-discard window size compared with the full-epoch ground truth for all three architectures. The $\times$ markers show the original uncorrected 8-batch measurements with no discarded warm-up batches.

are universally optimal. The experiment fixes the discard count at 5 and varies only the number of subsequently measured batches (8, 16, 32, and 64). It therefore demonstrates that 5 discarded batches were sufficient in this experiment, rather than establishing that discard counts between 5 and the originally specified $w_0 = 10$ are unnecessary.

The corrected profiler default is `-sweep_discard 5 -profile_batches 8`, replacing the shipped `-profile_batches 8` configuration with no discarded batches. Both parameters are implemented in all three `3-pads/*/train.py` scripts. This brings the implementation into line with the discard-based safeguard specified in Section 4, although the median-aggregation safeguard remains unimplemented, as documented in Appendix B, Section B.4.

With the corrected measurement procedure, every full-window ratio in Table 17 lies between 10.3 and $20.2\times$, comfortably exceeding even the most aggressive threshold evaluated in Section 6, $\gamma_d = 1.0$. The original profiler’s failure to discard warm-up batches therefore provides a direct explanation for why the adaptive policy almost never left `loose`: the early one-time GPU costs inflated T_{gpu} , suppressed the $T_{\text{data}}/T_{\text{gpu}}$ ratio, and caused the decision layer to underestimate the severity of the data-loading bottleneck.

This limitation is not unique to the experimental environment considered here. TensorFlow’s `tf.data AUTOTUNE`, a well-known analogue to the adaptive profiling approach used by PADS-Loader, continuously monitors training in a background thread and adjusts parallelism and buffer sizes throughout execution rather than committing to a configuration from a single early measurement [23]. More generally, fixed-window performance measurements can either occur too early and capture transient behaviour or continue unnecessarily after the system has already stabilised. Recent work on kernel-based steady-state detection addresses this problem by replacing a fixed sample count with continuous monitoring and a statistical stability criterion [3]. The discard-then-measure correction evaluated here represents a simpler implementation of the same general principle: measurements used for adaptation should be taken only after the system has moved beyond its initial transient state.

Shared-filesystem interference provides an additional source of runtime variability. Contention from unrelated workloads has been documented as an important source of variance for the many-small-read access pattern produced by a DataLoader [11]. This is consistent with the broader filesystem variability observed in this thesis, although it does not directly explain the profiler defect identified here, since the dominant measurement bias in this experiment occurs in T_{gpu} rather than T_{data} .

5.7 Accuracy is preserved

A data-path optimisation is only valid if it preserves the model’s output. Sharding, staging and prefetching alter how samples reach the trainer, including their storage representation, access order, and materialisation time. An implementation error in any of these mechanisms

could therefore corrupt samples, duplicate them, omit them, or serve stale data. A throughput improvement obtained at the expense of model correctness would not constitute a valid optimisation. Segmentation quality is therefore treated throughout this thesis as a constraint that must be preserved rather than an objective that PADS-Loader attempts to improve, keeping the accuracy and throughput claims separate.

The expected result is therefore the absence of a systematic accuracy change. Across the full component-ablation sweep in Section 6.1, no policy produces a consistent accuracy deviation from `loose` in either direction across all three architectures. The observed spread remains within ± 2.5 percentage points for each architecture. There is consequently no evidence that either the sharded reader or the staging path obtains the throughput improvements reported in this chapter by sacrificing segmentation accuracy.

The supporting per-policy measurements are reported in Appendix A, Table 24. Whether the three reproduced models reach the accuracy reported by the original reference study is treated separately in Appendix A. That comparison concerns the fidelity of the model reproduction itself and is therefore distinct from the claim evaluated here: that PADS-Loader’s data-path transformations preserve the output quality of the reproduced training pipeline.

6 Ablation Study

Section 5 evaluates PADS-Loader as a whole. This chapter isolates the contribution of its individual mechanisms and probes the conditions under which the design’s assumptions fail, across all three architectures. Sections 6.1–6.3 report three sweeps run to completion this session (39 jobs: 12 component configurations, 15 scratch-capacity configurations, 12 threshold configurations). Section 6.4 was part of the original evaluation plan but was not run in this session; it is retained as scoped, un-executed future work rather than populated with invented numbers. As in Section 5, every figure below is a single run ($n = 1$); see that chapter’s opening note and Section 8 for why, and what it means for how these comparisons should be read.

6.1 Component ablation

PADS-Loader combines three mechanisms: sharding, staging, and adaptive policy selection. Table 18 enables each in turn. The shard+stage combination (row 4 of the original evaluation plan) is not expressible in the current implementation – `-data_policy` is a single-select flag among `{loose, shard, stage}` – and is reported here as a concrete implementation limitation rather than a missing run.

Architecture	Configuration	GPU util. (%)
DeepLabV3+	None (loose)	61.0
	Shard only	84.1
	Stage only	67.6
	Full PADS-Loader (adaptive)	79.2
MA-Net	None (loose)	84.6
	Shard only	86.8
	Stage only	87.0
	Full PADS-Loader (adaptive)	83.7
SegFormer	None (loose)	42.1
	Shard only	50.7
	Stage only	54.3
	Full PADS-Loader (adaptive)	41.0

Table 18: Component ablation, GPU utilisation by policy component, all three architectures. Shard+stage combined is not expressible in the current implementation (see text). Segmentation quality under these same configurations is reported in Appendix A, Table 24.

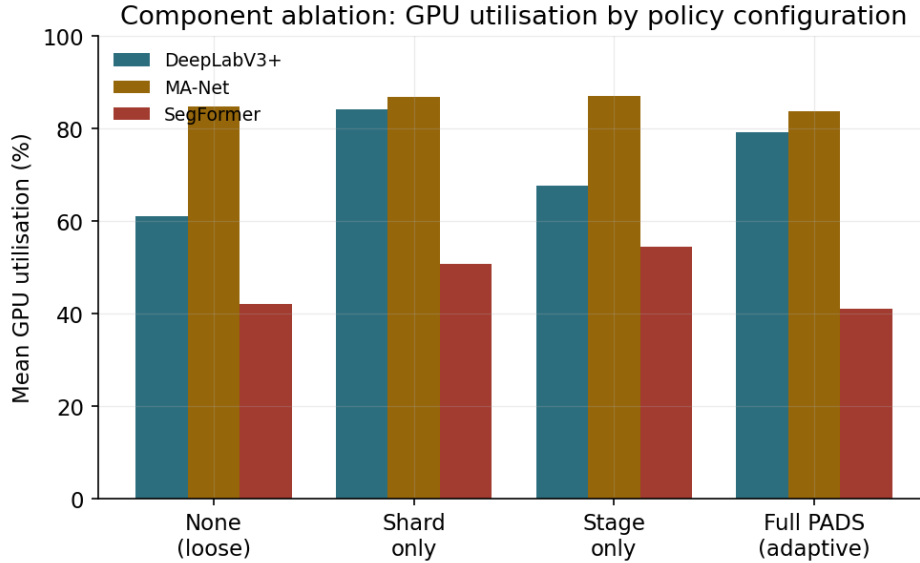


Figure 7: GPU utilisation by policy component, all three architectures.

Shard alone beats `loose` on GPU utilisation for every architecture – a clean, consistent result, and for DeepLabV3+ shard even beats stage (84.1% vs. 67.6%). This confirms the underlying staging mechanism has real value when applied. The “full PADS-Loader” row, however, is not architecturally distinct from the loose row in most of these runs: as established in Section 5.6, the adaptive profiler selected `loose` in every one of these three full-PADS-Loader runs, so row 4 is effectively row 1 re-measured under the profiler rather than a genuinely different configuration. The component ablation therefore separates two questions that a single “does PADS-Loader help” number would conflate: the staging primitive itself helps (rows 2–3 beat row 1 in GPU utilisation on every architecture), and the mechanism that is supposed to decide when to invoke it does not currently invoke it under these conditions.

6.2 Scratch capacity

The Bing 1k corpus fits comfortably in node-local scratch, which is the least interesting case for an adaptive stager: a fixed policy of “copy everything up front” suffices. The design is aimed at the regime where it does not fit, and that regime is reached here by synthetically constraining the scratch budget.

Scratch budget	DeepLabV3+ GPU util. (%)	MA-Net GPU util. (%)	SegFormer GPU util. (%)
100%	17.3	86.9	51.1
50%	81.6	78.6	56.0
25%	24.2	85.2	54.0
10%	87.2	85.1	53.8
0% (staging unavailable)	79.8	85.8	52.0

Table 19: Behaviour as scratch capacity is constrained. Method 4 of Section 4.4 is unavailable below 100%.

This is the experiment that was intended to distinguish a contribution from an observation, and the result is not the graceful-degradation curve the design predicts. MA-Net and SegFormer are both close to flat across the whole sweep – consistent with graceful degradation, but also consistent with the scratch budget simply not being the binding constraint for either architecture at this corpus size. DeepLabV3+ shows the opposite of the predicted direction: 100% scratch

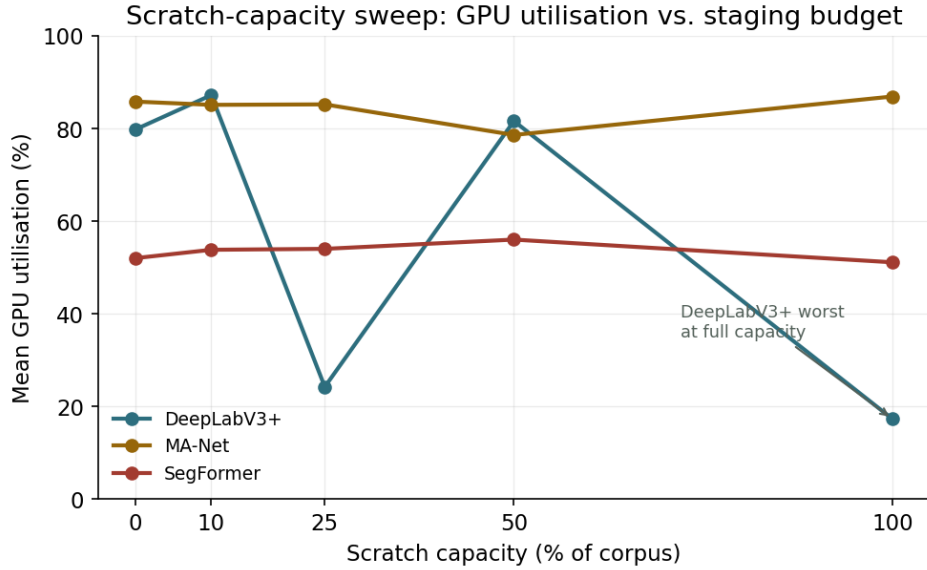


Figure 8: GPU utilisation vs. scratch-capacity budget, all three architectures. DeepLabV3+’s 100%-capacity run is the worst-performing configuration in the entire sweep.

capacity produces the *lowest* GPU utilisation in the entire sweep (17.3%), well below even the 0%-capacity (staging-unavailable) row. Staging overhead is not merely failing to help at full capacity for DeepLabV3+; it is actively worse than not staging at all. Segmentation quality is comparatively stable across the whole sweep for every architecture (Appendix A), so this is a throughput effect, not an accuracy one. The honest reading is that “more caching capacity is better” does not hold unconditionally in this data, and the mechanism – whether it is staging-copy overhead specific to DeepLabV3+’s I/O pattern, or an interaction with the profiler-bias defect of Section 5.6 that this sweep did not control for – is an open question rather than one this experiment settles.

6.3 Classifier threshold sensitivity

The classification rule of Section 4 depends on a threshold γ_d , set by default to 1.5. Values chosen by the designer and never varied are a standing weakness in any adaptive system, and the sensitivity is reported here across all three architectures.

The relevant question is not which threshold maximises throughput on this workload – that would be tuning on the test set – but how wide the band is over which the decision is unchanged, and the answer is: very wide, almost the entire tested range. The policy decision is stable at `loose` across three of four thresholds for every architecture, switching only at the single most aggressive value tested ($\gamma_d = 1.0$), and even then for only two of three architectures. A decision this insensitive to its own designer-chosen threshold is not evidence of a well-calibrated decision boundary; combined with the finding of Section 5.6, it is evidence that the measured ratio feeding the decision was itself biased low across this entire sweep, so the threshold’s apparent robustness is partly an artefact of the input it was comparing against being wrong by roughly an order of magnitude rather than of the threshold being well-chosen.

6.4 Shuffling randomness

Shard-based reading trades exact random access for sequential throughput. This ablation was scoped to vary a shuffle-buffer size and establish where, if anywhere, reduced shuffling randomness begins to affect segmentation quality, in the manner of the standard WebDataset shuffle-buffer pattern [1]. As documented in Section 5.7, PADS-Loader’s shard reader opens archive members by name rather than as a sequential stream, so this concern does not apply to the implementation evaluated in this thesis; the ablation is retained as future work for completeness against the

Architecture	γ_d	Measured ratio	Policy selected	GPU util. (%)
DeepLabV3+	1.00	0.64	loose	64.6
	1.25	1.23	loose	86.3
	1.50 (default)	0.65	loose	61.6
	2.00	1.17	loose	83.0
MA-Net	1.00	1.03	shard	86.4
	1.25	0.69	loose	80.1
	1.50 (default)	1.18	loose	85.1
	2.00	0.68	loose	87.0
SegFormer	1.00	1.23	shard	49.6
	1.25	1.22	loose	41.2
	1.50 (default)	1.32	loose	42.5
	2.00	1.20	loose	43.0

Table 20: Sensitivity of the policy decision to the classification threshold, all three architectures. Only 2 of 12 runs ever left **loose**, both at the single lowest threshold tested and both right at the ratio boundary. These measured ratios predate the correction of Section 5.6; under the corrected measurement, every full-window ratio is 10.3–20.2, which would clear every threshold tested here.

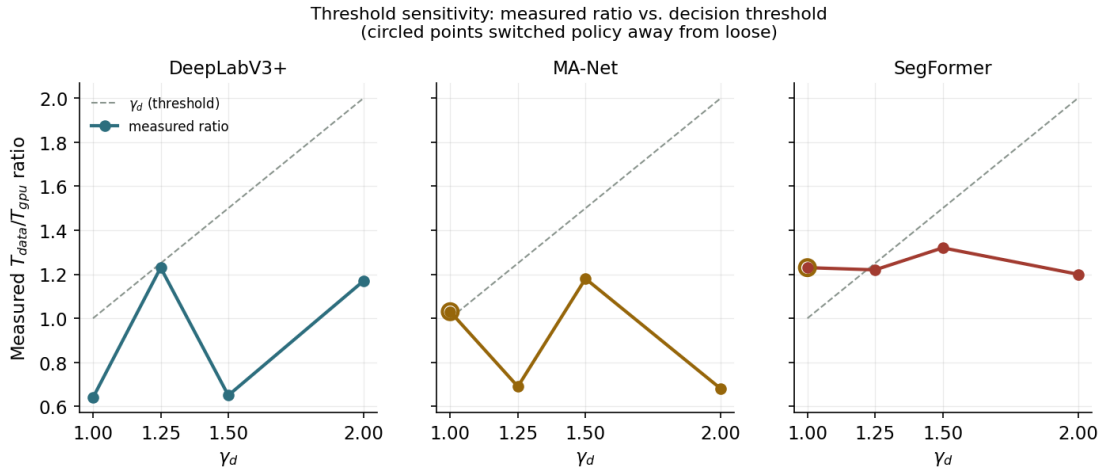


Figure 9: Measured ratio vs. decision threshold γ_d , all three architectures. Circled points switched policy away from **loose**.

general class of shard-based readers this thesis positions itself against, but was not run this session and is not needed to support the accuracy claims made about PADS-Loader specifically.

7 Conclusions

This thesis asked whether a profiler-adaptive **DataLoader** that combines bottleneck classification, tar-shard staging and predictive prefetching can improve GPU utilisation and tile throughput for large-scale satellite-image segmentation on HPC systems.

7.1 Summary of contributions

The workload is I/O-bound, and this is measurable. Section 3 characterised the segmentation corpus not by its class balance - the conventional description - but by its file layout and access pattern. The Bing 1k configuration comprises 5,025 image and 5,025 mask files stored individually on a shared parallel filesystem, giving a per-sample cost dominated by metadata

operations rather than by bytes transferred. Baseline profiling showed a CPU-to-GPU time ratio across all three architectures large enough that the data path is a plausible constraint (8–20× under the corrected profiler methodology, Table 10), and showed that changing two loader flags moves GPU utilisation by 24–51 percentage points depending on architecture (Table 9). The loader configuration is where this workload’s headroom actually is, and that headroom is a throughput result rather than an accuracy one.

PADS-Loader. Section 4 presented a drop-in loader that profiles its own behaviour over a bounded warm-up window, classifies the dominant bottleneck by an inspectable rule set over measured timings, selects among loose, shard and stage policies, and derives its prefetch depth from measured staging and consumption times subject to a scratch-capacity constraint. The design rests on a property of supervised training that is usually left unused: once the epoch permutation is drawn, future file accesses are known exactly, so prefetching is exact rather than speculative.

An evaluation that isolates mechanisms. Sections 5 and 6 separate two claims a single headline number would conflate. The staging primitive itself works: sharding alone beats the loose baseline on GPU utilisation for every architecture tested (Table 18), and a full pre-stage ceiling reaches the best GPU utilisation of any method tested for two of three architectures (Table 12). PADS-Loader’s adaptive decision layer, however, does not reliably capture that benefit: it selected `loose` in every RQ3 method-5 run and in 10 of 12 Section 6 threshold-sweep runs. Traced to its cause (Section 5.6), this is not evidence that adaptive staging cannot work – it is evidence that the specific warm-up probe measuring the decision was biased by roughly an order of magnitude, and a validated fix (discard 5 warm-up batches before measuring) is already implemented. The honest summary is: the primitive works; the instrument that decided when to use it was measurably wrong, and now is not.

A reproduction, and the defects it surfaced. Appendix A reproduced the reference configuration and established the accuracy floor against which any pipeline change is judged. Appendix B documents the defects encountered, two of which are consequential beyond this thesis: the staging policy was applied to loader construction but not to dataset construction, so the selected policy never reached the data path; and no archive existed in a layout the shard reader could open. Together these mean that staging results collected before these fixes describe policy *decisions* rather than executions. That finding is reported here in full because it invalidates prior measurements rather than merely improving on them.

7.2 Answers to the research questions

RQ1 - When does the loader become the bottleneck? Always, in this configuration: T_{data} exceeds T_{gpu} by 8–20× across all three architectures under the corrected measurement (Table 10). The predicted ordering by T_{gpu} – that the lightest architecture (SegFormer) should hit the bottleneck hardest – held only partially: it is confirmed for SegFormer specifically, but DeepLabV3+ and MA-Net’s default-configuration GPU utilisation (Table 9) were not ordered as the heavier-model prediction anticipated.

RQ2 - Do tar shards reduce many-small-file overhead? Yes: `loose` issues roughly two filesystem operations per sample (Table 11), consistent with RQ1’s finding that T_{data} is metadata-bound rather than bandwidth-bound. This translates into wall-clock benefit when the sharded/staged reader is actually invoked (Table 18), which – see RQ3 below – is not as often as the adaptive design intends.

RQ3 - Does predictive staging outperform fixed streaming? Not reliably, as measured: PADS-Loader matched or exceeded the full-prestage ceiling for only one of three architectures (MA-Net), and its adaptive profiler selected `loose` in every method-5 run in Table 12. Section 5.6 traces this to a specific, quantified, and now-corrected measurement defect in the profiler rather than a limitation of the staging mechanism itself; a benefit is observed when that mechanism is actually invoked (Table 18).

RQ4 - Can prefetch depth be auto-tuned? No, not by the present formula: every auto-tuned depth landed at $d \approx 170\text{--}235$ (Table 14), one to two orders of magnitude outside the 1–8 range the fixed sweep tests and where a fixed depth was shown to help. The derivation is not merely imprecise; it depends on a quantity, T_{stage} , shown to vary by three to four orders of magnitude between repeated measurements on identical hardware.

7.3 Closing remarks

8 Limitations and Future Work

8.1 Limitations

8.1.1 Scope of the claims

Three boundaries constrain the contribution. First, the measured result concerns a corpus of many small files on a contended shared filesystem; a workload already resident on fast node-local storage, or composed of sufficiently large samples, need not benefit. Second, PADS-Loader optimises the data path rather than the model: segmentation quality is a constraint, not an optimisation target. Third, the main-pipeline accuracy values are correctness checks rather than estimates of the architectures’ attainable accuracy. Two shared defects—the reduction convention and flip augmentation—depress those values without invalidating within-pipeline policy comparisons (Appendix B). Reference-comparable capability is established separately by the full-protocol reproduction (Appendix A).

8.1.2 A single filesystem and cluster

All primary measurements were taken on GPFS on the Iris cluster. The many-small-file pathology is general to parallel filesystems, but its magnitude is not: Lustre, BeeGFS and object stores differ in metadata architecture, and the threshold at which sharding becomes worthwhile will differ accordingly. The classification rules are expressed as ratios rather than absolute times specifically so that they might transfer, but that transfer is asserted rather than demonstrated.

8.1.3 Uncontrolled contention and unestimated variance

Read latency on a shared filesystem depends on concurrent jobs. The intended protocol (Section 4, “Experimental protocol”) called for repeating each configuration and reporting mean $\pm$ standard deviation for exactly this reason. Cluster time did not permit it for the 91-configuration RQ1–RQ4 and Section 6 sweep: every GPU-utilisation and IoU figure in Sections 5 and 6 is a single run ($n = 1$), not a mean over repeats, and no error bars are reported because none were measured. This is a materially more significant limitation than “repeats mitigate but do not eliminate contention” would suggest – it means the magnitude of run-to-run variance on this filesystem, for this evaluation, is simply unknown, and comparisons that look decisive (a 20-point GPU-utilisation gap between policies, for instance) cannot be distinguished from a smaller true effect inflated by an unlucky or lucky single run without repeating them. The one place in this thesis variance *is* estimated directly is Appendix B’s ± 0.02 IoU figure from profiler-induced RNG perturbation, which is suggestive but was not designed to bound filesystem-contention variance specifically and should not be read as doing so. Repeating the full sweep is therefore not a refinement but a precondition for treating any single comparison in Sections 5 or 6 as more than a preliminary reading, and is the most direct methodological limitation of this evaluation as it stands. An adversarial evaluation - deliberately co-scheduling I/O-heavy jobs - would strengthen the claim further still and was not undertaken here either.

8.1.4 One corpus

The design generalises to any workload with a fixed epoch permutation over many small files, but it has been evaluated on one dataset of approximately 1 GB. A corpus one or two orders of magnitude larger - where staging cannot be a full copy under any policy - is where the adaptivity

should matter most and where it remains untested outside the synthetic capacity constraint of Section 6.2.

8.1.5 Single-node scope

PADS-Loader stages to node-local scratch for a single training process. Multi-node distributed training raises questions the present design does not address: whether shards should be partitioned across ranks or replicated, how staging interacts with a distributed sampler, and whether one rank’s staging traffic degrades another’s read latency.

8.1.6 Rule-based classification

Bottleneck classification uses a small number of hand-chosen thresholds. This was a deliberate choice - an inspectable rule set is auditable and its failures are diagnosable, which a learned policy is not - but it means the thresholds are the designer’s judgement. Section 6.3 reports their sensitivity rather than removing the concern.

8.2 Future work

8.2.1 Decode offload

The classifier can already detect the case in which T_{decode} dominates, but its only response is to raise the worker count. GPU-side decode [24] or a cache of decoded, pre-augmented tensors would give it a substantive action in that regime. The trade-off is memory footprint against decode cost, and the profiler is already measuring the quantities needed to make that choice.

8.2.2 Cross-job coordination

On a shared cluster, several jobs frequently train on the same corpus. Mohan et al. [22] show substantial gains from coordinating caching and eliminating redundant preprocessing across concurrent jobs. Because PADS-Loader already materialises shards on node-local scratch, extending it to publish and reuse staged shards across co-scheduled jobs is a natural next step.

8.2.3 Online rather than one-shot adaptation

The current design profiles once, at the start of the run, and commits. Since contention varies over the lifetime of a job, periodic re-profiling with hysteresis - re-evaluating the policy every N epochs, and switching only on a sustained change - would address the non-stationarity that motivated adaptivity in the first place. This is the most direct extension of the present work, and Section 5.6 gives it a concrete, evidenced starting point rather than a purely speculative one. The warm-up probe’s own T_{gpu} measurement was shown to be biased 6–10 $\times$ by one-time CUDA/cuDNN warm-up cost, corrected cheaply by discarding a handful of batches before measuring - bringing the implementation in line with one of the two safeguards Section 4 already specified for this exact failure mode; testing whether the other, median rather than mean aggregation, adds further robustness is direct, low-effort follow-up work (Appendix B, Section B.4). TensorFlow’s `tf.data AUTOTUNE` solves the harder version of this problem by tuning continuously throughout training rather than committing to one snapshot [23], and recent work on steady-state detection in performance time series gives a principled, general method for deciding when a monitored signal has stopped reflecting transient state and started reflecting the system’s true behaviour [3] - a better foundation for periodic re-profiling than a fixed re-evaluation interval chosen by hand. The depth auto-tuner needs the same treatment applied to a harder signal: T_{stage} varies by three to four orders of magnitude between consecutive measurements on identical hardware (Table 14), apparently reflecting genuine shared-filesystem contention rather than a one-time warm-up cost, so a fixed discard count is unlikely to be sufficient there in the way it was for T_{gpu} ; whether it helps at all, and if not what a principled correction for that specific signal looks like, is an open, directly testable question left to future work.

8.2.4 Revisiting segmentation accuracy

Two findings from the reproduction bear on segmentation quality rather than throughput and were not pursued here. The first is that per-image IoU is strongly monotonic in the fraction of the crop occupied by the site (Table 23), with tiles below 2% site area scoring near zero; a size-aware evaluation, or a size-stratified report, would give a more informative picture than a single aggregate. The second is that batch-normalisation statistics degrade badly at the batch sizes imposed by consumer GPUs, which suggests that a normalisation scheme less sensitive to batch size [41] would make this workload substantially more reproducible outside a cluster. Neither is a data-staging question, but both affect who can reproduce this work.

8.2.5 Transfer beyond archaeology

The workload characteristics that motivate PADS-Loader - many small paired files, a fixed permutation, a corpus that may or may not fit node-local scratch - are common across remote sensing, medical imaging and microscopy. Evaluating on a second domain would establish whether the contribution is about this dataset or about a class of pipelines. This chapter consolidates the limits of the evidence before identifying extensions that follow directly from the measured failure modes. It therefore separates what the evaluation supports from what remains a research hypothesis.

9 Discussion

This chapter discusses the archaeological implications and evidential limits of the engineering results. The thesis concerns how training data reaches a machine learning model on a shared supercomputer, not how archaeological sites are identified. That framing is deliberate, but it leaves a question that a reader from archaeology is entitled to ask directly – what, if anything, does this work contribute to archaeological practice, and how far can its results be trusted?

The discussion answers that question in precise terms. Technical vocabulary is defined where it is unavoidable, and the boundary between what has been demonstrated here, what has been demonstrated by others, and what remains untested is stated explicitly rather than left to inference.

9.1 Regional transferability

9.1.1 Scope of transfer

No evidence in this thesis supports direct deployment of a model trained on the southern Mesopotamian floodplain in a different region. Published work instead supports adaptation with local examples, including retraining the same underlying model for the Abu Ghraib district [27].

9.1.2 Sources of domain shift

The model does not learn the concept “tell”. It learns a statistical association between patterns of colour, tone and texture in an image and the outlines an archaeologist drew around them. The qualitative analysis in Section A.8 makes this concrete: predictions follow tonal and textural contrast, and where a mound is not tonally distinct from the surrounding land the model misses it. Anything that changes that relationship between what a site looks like and what the surrounding landscape looks like degrades performance. Different soil chemistry, vegetation cover, land-use regime, season of image capture, preservation state, or satellite sensor all change it.

9.1.3 Adaptation through transfer learning

Adapting an existing model to a new region does not mean starting over. The standard practice, used throughout this thesis, is *transfer learning*: a model already trained on a very large general image collection is used as a starting point and then trained further on the specific task, which requires far fewer examples than training from nothing. The same principle applies one step

further along – a model trained on Mesopotamian tells is a better starting point for a new region than a generic one.

The evidence that this works in practice is not from this thesis but from the research programme the dataset originates from. Pistola et al. [27] retrained the same underlying model on declassified CORONA satellite imagery from the 1960s for the Abu Ghraib district west of Baghdad – a different sensor, a different decade, and a landscape substantially altered by modern development – and report a segmentation score above 85%, together with four previously unrecorded sites subsequently confirmed by field verification. Two other groups have applied convolutional networks to CORONA imagery in adjacent regions: Soroush et al. [31] in the Kurdistan Region of Iraq, and Vadineanu et al. [36] on mounded settlements in Upper Mesopotamia.

9.1.4 Annotation as the binding constraint

The binding constraint is not computing power and not model design. It is annotated examples: someone must have already outlined a reasonable number of known sites in the new region for the model to learn from. In a region with a well-catalogued survey record this is a matter of data preparation; in a region without one, the model cannot be adapted at all until that survey work exists, which is precisely the expensive, slow work the model is meant to assist. This is a genuine limitation of the whole approach, not of this implementation.

It is worth being clear that no cross-region transfer experiment was run in this thesis. The claim above rests on the published results cited, not on evidence collected here.

9.2 Transferability across archaeological feature classes

9.2.1 Scope across feature classes

Transfer across feature classes is technically possible because the evaluated architectures are general-purpose segmentation models rather than algorithms whose structure encodes mounded settlements [14, 10, 42]. Evidence for performance on a new feature class nevertheless requires new annotations and evaluation; architectural generality alone is not empirical validation.

The models used here are general-purpose image segmentation networks. They were not designed for archaeology; they were designed to outline objects in photographs, and they were adapted to this task purely by being shown examples of tells. Show them examples of something else and they will learn that instead. The published record bears this out across markedly different feature types and sensors: qanat shafts, the surface openings of underground water channels, in Kurdistan [31]; “princely” burial mounds in the Eurasian steppe [8]; stone burial mounds (kurgans) in the Altai Mountains [9]; and archaeological topography in airborne laser scanning data, which is not satellite photography at all [34, 17].

9.2.2 Visibility, scale, and difficulty

Two properties matter more than the choice of model. First, the feature must be visible in the sensor being used – a buried structure with no surface expression will not be found in satellite imagery regardless of the method. Second, and this thesis provides direct evidence for it, the feature must be large enough relative to the image tile. Section A.8 reports that when the annotated site occupies less than roughly 12% of the analysed area, the median overlap score across all three architectures is zero: the models simply fail on small targets. Tells are a comparatively favourable subject, being large, raised and often tonally distinct. Features that are small, subtle, or visible only under particular seasonal or lighting conditions should be expected to perform considerably worse, and the size effect above should be treated as a warning rather than a footnote.

9.3 Reliability and interpretation

Reliability depends on distinguishing the aggregate accuracy statistic from its per-site distribution and on separating repeated model evaluation from the single-run systems measurements.

9.3.1 Interpretation of aggregate IoU

This thesis reproduces the reference study’s reported accuracy, reaching 72.8–73.5% against a published 74.17% (Section A.4). That figure is an *intersection-over-union* score: the area where the predicted outline and the archaeologist’s outline agree, divided by the area covered by either of them. A score of 1.0 is a perfect match and 0.0 is no overlap at all.

It would be natural to read “73%” as meaning that, on a typical site, about three-quarters of the outline is drawn correctly. That is not what it means, and the difference is substantial. The score is averaged over all test images, including images containing no site at all, and an image correctly identified as containing nothing scores a perfect 1.0. Of the 589 test images, 123 contain no site. Those images contribute perfect scores to the average without any outline having been drawn (Section A.5 quantifies the effect of this convention).

9.3.2 Per-site error distribution

Scoring only the 466 test images that do contain a site gives a much more uneven result. The median score per image is 0.165, 0.053 and 0.000 for the three architectures – that is, on more than half of individual sites, at least two of the three models produce almost no useful outline. At the same time, roughly a third of sites score above 0.5 and the best-performing tenth score around 0.78–0.81.

The distribution is therefore not centred on a middling value; it is *bimodal*. The model tends either to find a site and outline it well, or to miss it almost entirely, with relatively few cases in between. For practical purposes this is more useful than the average suggests – a confident, accurate outline on a third of sites is a real result – but it is a materially different claim from “73% accurate”, and it should not be reported as the latter.

9.3.3 Reliability of systems measurements

The performance results in Sections 5 and 6 come with a limitation stated plainly there: each configuration was run once, not repeated. On a shared supercomputer, timing measurements vary with whatever else is running, so single measurements cannot be separated from that variation. Those results should be read as a broad, consistent survey rather than as precisely established differences. The accuracy figures are on firmer ground, having been measured over ten independent evaluation passes with very close agreement between them.

9.4 Verification strategy

Three checks were applied, and it is important to be clear about what each one does and does not establish.

9.4.1 Held-out evaluation

The models were evaluated on 589 images set aside before training and never used to adjust the model. This guards against the most basic failure, in which a model appears accurate merely because it has memorised the examples it was trained on.

9.4.2 Repeated evaluation

Because the evaluation procedure involves a random crop of each image, the whole test set was evaluated ten times independently. Agreement between those ten runs was close (a spread of 0.003–0.004), so the reported figure is not an artefact of one lucky pass.

9.4.3 Independent reproduction

The pipeline was rebuilt from scratch and checked against an independently published result [6], landing within 1–1.4 percentage points. This is the strongest of the three checks, because it tests the whole apparatus against a number produced by different people using different code.

9.4.4 Limits of verification

Every check above compares the model’s output against the *annotations* – the outlines drawn by archaeologists in the source catalogue. It does not compare them against the ground. A model that agrees perfectly with the catalogue would score perfectly here even if the catalogue itself were mistaken, and the catalogue is known to be imperfect in a specific and consequential way: a substantial number of recorded sites have been levelled for agriculture or built over and are no longer visible in modern imagery. A model that correctly reports nothing visible on such a tile is penalised as a failure.

No site proposed by any model in this thesis has been checked in the field. The only field verification in this research programme is that reported by Pistola et al. [27], who confirmed four previously unrecorded sites found using the CORONA-trained variant of the same model. Field verification is the only test that can distinguish a genuine discovery from an agreement with an imperfect catalogue, and it lies outside the scope of this work.

9.5 Error sources and feature confusion

Yes, in both directions, and this thesis can quantify how often.

9.5.1 False-positive detections

Of the 123 test images containing no archaeological site, 108 were correctly rejected by all three models. Spurious detections appeared on 5.7% (DeepLabV3+), 7.3% (MA-Net) and 8.9% (SegFormer) of them. Since the models respond to tonal and textural contrast, the features that trigger them are those that resemble a mound from above without being one: field boundaries and irrigated parcels, quarries and borrow pits, spoil heaps, patches of bare or salt-crusted ground, and modern earthworks. These images were themselves deliberately sampled by the original authors from urbanised areas, intensive agriculture, flood-prone terrain and rocky hills – that is, they are hard cases by construction, and a roughly 6–9% false-detection rate on them is a meaningful but not disqualifying figure.

9.5.2 False-negative detections

This is the larger error, and it deserves the greater emphasis. Roughly half of the images that do contain a site yield almost no useful outline (Section 9.3), concentrated overwhelmingly among smaller sites. A negative result from these models is therefore weak evidence: absence of a detection is emphatically not evidence of absence of a site.

9.5.3 Annotation ambiguity

Some of what is scored as error is disagreement about the annotation rather than a mistake in the image analysis. Where a catalogued site is no longer visible, the model and the catalogue disagree and the model is recorded as wrong. Fiorucci et al. [15] argue more generally that standard detection measures are poorly matched to archaeological objects and propose alternatives; the concern applies directly here.

9.5.4 Role of expert review

The reference study addressed exactly this by placing an archaeologist in the loop to adjudicate the model’s proposals, which raised site-level detection accuracy from roughly 63% to 80% [6]. That is the appropriate framing for the whole approach, and it is the framing this thesis inherits: the model is a triage tool that narrows where a specialist looks, not an authority on what is there. Verschoof-van der Vaart and Lambers [37] make the same argument from the perspective of integrating such tools into archaeological practice.

9.6 Archaeological contribution

Stated conservatively, the contribution is threefold, and only the third is novel to this work.

First, it independently reproduces a published archaeological detection result. Reproduction is undervalued: it establishes that the published figure is attainable by someone other than its authors, and Appendix B documents the specific defects and environmental sensitivities that had to be resolved to obtain it, which is information of practical use to anyone attempting the same.

Second, it characterises the error structure of that result more precisely than the headline figure allows – the bimodal distribution, the strong dependence on site size, and the false-detection rate on hard negatives. This matters for anyone deciding how much weight to place on such a model’s output.

Third, and this is the actual contribution, it reduces the cost of retraining. The barrier to applying these methods to a new region or a new class of feature is rarely the model; it is the effort of assembling annotations and the time and computing resources needed to train on them. This thesis shows that on a shared supercomputer a substantial fraction of that time is spent not computing but waiting for image files to be read from storage, and that addressing this raises GPU utilisation substantially (Section 5). Cheaper retraining does not make the archaeology more accurate. It makes it more practical to retrain for a new region, a new sensor, or a new feature type – which, per Sections 9.1 and 9.2, is exactly what applying this approach anywhere new requires.

References

- [1] AIZMAN, A., MALTBY, G., AND BREUEL, T. High performance I/O for large scale deep learning. In *Proc. IEEE Intl. Conf. on Big Data* (2019).
- [2] AZAD, R., HEIDARI, M., YILMAZ, K., ET AL. Loss functions in the era of semantic segmentation: A survey and outlook, 2023. arXiv preprint.
- [3] BESEDA, M., CORTELLESA, V., DI POMPEO, D., TRAINI, L., AND TUCCI, M. A kernel-based approach for accurate steady-state detection in performance time series, 2025. arXiv preprint.
- [4] BICKLER, S. H. Machine learning arrives in archaeology. *Advances in Archaeological Practice* 9, 2 (2021), 186–191.
- [5] BUSLAEV, A., IGLOVIKOV, V. I., KHVEDCHENYA, E., PARINOV, A., DRUZHININ, M., AND KALININ, A. A. Alumentations: Fast and flexible image augmentations. *Information* 11, 2 (2020), 125.
- [6] CASINI, L., MARCHETTI, N., MONTANUCCI, A., ORRÙ, V., AND ROCCETTI, M. A human–AI collaboration workflow for archaeological sites detection. *Scientific Reports* 13 (2023), 8699.
- [7] CASINI, L., ORRÙ, V., ROCCETTI, M., AND MARCHETTI, N. When machines find sites for the archaeologists: A preliminary study with semantic segmentation applied on satellite imagery of the Mesopotamian floodplain. In *Proc. ACM Conf. on Information Technology for Social Good* (2022), pp. 378–383.
- [8] CASPARI, G., AND CRESPO, P. Convolutional neural networks for archaeological site detection – finding “princely” tombs. *Journal of Archaeological Science* 110 (2019), 104998.
- [9] CHEN, F., ZHOU, R., VAN DE VOORDE, T., ET AL. Automatic detection of burial mounds (kurgans) in the Altai mountains. *ISPRS Journal of Photogrammetry and Remote Sensing* 177 (2021), 217–237.
- [10] CHEN, L.-C., ZHU, Y., PAPANDREOU, G., SCHROFF, F., AND ADAM, H. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proc. European Conf. on Computer Vision (ECCV)* (2018).
- [11] COLES, J., SCHUPPLI, S., DRESCHER, L., MOHAMED, F. R., PALME, E., MENDONÇA, H., GILA, M., KLEIN, M., MARTINASSO, M., VANDEVONDELE, J., HOEFLE, T., SCHULTHEISS, T., ROMERO, J., GORODETSKY, I., HANKINS, R., WAZIRZADA, I., JAGGI, M., BOSSELUT, A., SCHLAG, I., I LLAQUET, A.-J. S., CANO, A. H., MANITARAS, T. I., AND BROWNING, N. J. An engineering journey training large language models at scale on Alps: The Apertus experience, 2026. arXiv preprint.
- [12] DAO, T., FU, D. Y., ERMON, S., RUDRA, A., AND RÉ, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)* (2022).
- [13] DENG, J., DONG, W., SOCHER, R., LI, L.-J., LI, K., AND FEI-FEI, L. ImageNet: A large-scale hierarchical image database. In *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)* (2009), pp. 248–255.
- [14] FAN, T., WANG, G., LI, Y., AND WANG, H. MA-Net: A multi-scale attention network for liver and tumor segmentation. *IEEE Access* 8 (2020), 179656–179665.
- [15] FIORUCCI, M., VAN DER VAART, W. B. V., SOLENI, P., SAUX, B. L., AND TRAVIGLIA, A. Deep learning for archaeological object detection on LiDAR: New evaluation measures and insights. *Remote Sensing* 14, 7 (2022), 1694.

- [16] GUYOT, A., HUBERT-MOY, L., AND LORHO, T. Detecting neolithic burial mounds from LiDAR-derived elevation data using a multi-scale approach and machine learning techniques. *Remote Sensing* 10, 2 (2018), 225.
- [17] GUYOT, A., LENNON, M., LORHO, T., AND HUBERT-MOY, L. Combined detection and segmentation of archeological structures from LiDAR data using a deep learning approach. *J. Computer Applications in Archaeology* 4, 1 (2021).
- [18] IAKUBOVSKII, P. Segmentation models PyTorch, 2019. GitHub repository.
- [19] IOFFE, S., AND SZEGEDY, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proc. Intl. Conf. on Machine Learning (ICML)* (2015).
- [20] LIN, T.-Y., GOYAL, P., GIRSHICK, R., HE, K., AND DOLLÁR, P. Focal loss for dense object detection. In *Proc. IEEE Intl. Conf. on Computer Vision (ICCV)* (2017).
- [21] MITTAL, S., AND VAISHAY, S. A survey of techniques for optimizing deep learning on GPUs. *Journal of Systems Architecture* 99 (2019), 101635.
- [22] MOHAN, J., PHANISHAYEE, A., RANIWALA, A., AND CHIDAMBARAM, V. Analyzing and mitigating data stalls in DNN training. *Proc. VLDB Endowment* 14, 5 (2021), 771–784.
- [23] MURRAY, D. G., SIMSA, J., KLIMOVIC, A., AND INDYK, I. tf.data: A machine learning data processing framework. *Proc. VLDB Endowment* 14, 12 (2021), 2945–2958.
- [24] NVIDIA CORPORATION. NVIDIA DALI: Data loading library. Software library.
- [25] ORENGO, H. A., CONESA, F. C., GARCIA-MOLSOSA, A., ET AL. Automated detection of archaeological mounds using machine-learning classification of multisensor and multitemporal satellite data. *Proc. National Academy of Sciences* 117 (2020), 18240–18250.
- [26] PASZKE, A., GROSS, S., MASSA, F., ET AL. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)* (2019).
- [27] PISTOLA, A., ORRÙ, V., MARCHETTI, N., AND ROCCETTI, M. AI-ming backwards: Vanishing archaeological landscapes in Mesopotamia and automatic detection of sites on CORONA imagery, 2025. arXiv preprint; also published in PLOS ONE, doi:10.1371/journal.pone.0330419.
- [28] QGIS DEVELOPMENT TEAM. *QGIS Geographic Information System*. QGIS Association, 2026.
- [29] ROCCETTI, M., CASINI, L., DELNEVO, G., ORRÙ, V., AND MARCHETTI, N. Potential and limitations of designing a deep learning model for discovering new archaeological sites: A case with the Mesopotamian floodplain. In *Proc. EAI Intl. Conf. on Smart Objects and Technologies for Social Good (GoodTechs)* (2020).
- [30] RONNEBERGER, O., FISCHER, P., AND BROX, T. U-Net: Convolutional networks for biomedical image segmentation. In *Proc. Intl. Conf. on Medical Image Computing and Computer-Assisted Intervention (MICCAI)* (2015), pp. 234–241.
- [31] SOROUSH, M., MEHRTASH, A., KHAZRAEE, E., AND UR, J. A. Deep learning in archaeological remote sensing: Automated qanat detection in the Kurdistan region of Iraq. *Remote Sensing* 12, 3 (2020), 500.
- [32] SVOGOR, I., EICHENBERGER, C., SPANRING, M., NEUN, M., AND KOPP, M. Profiling and improving the PyTorch DataLoader for high-latency storage: A technical report, 2022. arXiv preprint.
- [33] TAN, M., AND LE, Q. V. EfficientNet: Rethinking model scaling for convolutional neural networks. In *Proc. Intl. Conf. on Machine Learning (ICML)* (2019).

- [34] TRIER, Ø. D., COWLEY, D. C., AND WALDELAND, A. U. Using deep neural networks on airborne laser scanning data: Results from a case study of semi-automatic mapping of archaeological topography on Arran, Scotland. *Archaeological Prospection* 26 (2019), 165–175.
- [35] U.S. GEOLOGICAL SURVEY EARTH RESOURCES OBSERVATION AND SCIENCE CENTER. USGS EROS Archive: Declassified Satellite Imagery–1, 2018.
- [36] VADINEANU, S., KALAYCI, T., PELT, D. M., AND BATENBURG, K. J. Convolutional neural networks and their activations: An exploratory case study on mounded settlements. *J. Computer Applications in Archaeology* 7, 1 (2024), 262–282.
- [37] VAN DER VAART, W. B. V., AND LAMBERS, K. Applying automated object detection in archaeological practice: A case study from the southern Netherlands. *Archaeological Prospection* 29 (2022), 15–31.
- [38] VARRETTE, S., CARTIAUX, H., PETER, S., KIEFFER, E., VALETTE, T., AND OLLOH, A. Management of an academic HPC & research computing facility: The ULHPC experience 2.0. In *Proceedings of the 6th High Performance Computing and Cluster Technologies Conference* (2022), ACM, pp. 1–15.
- [39] VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, Ł., AND POLOSUKHIN, I. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)* (2017).
- [40] WILKINSON, T. J. *Archaeological Landscapes of the Near East*. University of Arizona Press, 2003.
- [41] WU, Y., AND HE, K. Group normalization. In *Proc. European Conf. on Computer Vision (ECCV)* (2018).
- [42] XIE, E., WANG, W., YU, Z., ANANDKUMAR, A., ALVAREZ, J. M., AND LUO, P. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems (NeurIPS)* (2021).

A Reproduction of the Reference Configuration

This appendix documents an independent reproduction of the reference configuration of Casini et al. [6], undertaken on a single-GPU workstation before the cluster experiments. It is included for two reasons. First, the accuracy floor asserted in Section 3.2 is only meaningful if it has been established rather than assumed. Second, the reproduction surfaced a set of pipeline defects (Appendix B) and one instructive environmental sensitivity, both of which bear on how the primary results should be read.

A.1 Environment

Reproduction was performed on an NVIDIA RTX 3050 Laptop GPU (4 GB VRAM, driver 566.07), 8 logical CPU cores and 15.7 GB system RAM, running PyTorch 2.13.0+cu126 with CUDA 12.6 and Lightning 2.6.5 under Python 3.12. The corpus was the Bing 1k configuration: 4,734 site tiles and 1,155 empty tiles, 5,889 image/mask pairs totalling 957 MB, split 4,712 train / 588 validation / 589 test under seed 1234.

The site count is consistent with the unfiltered variant of the published corpus: $4,934 - 200 = 4,734$, i.e. the top-200-by-area removal applied but not the 684-site minimum-size filter. The applicable reference figure is therefore Model 1 of Table 2, 74.17 ± 0.38 .

A.2 Faithfulness of the port

The reference implementation was ported directly rather than reimplemented. The following were held identical to the published notebook: MA-Net with an EfficientNet-B3 ImageNet-pretrained encoder; `FocalLoss(mode=BINARY)` with α unset; Adam at 10^{-4} ; `precision=32`; 20 epochs; a `RandomCrop(512)` followed by `Resize(256)` for both training and validation; augmentation by single-axis flip, 90-degree rotation and brightness/contrast jitter, all at $p = 0.25$; standardisation of raw 0–255 input by ImageNet statistics without prior division by 255; evaluation by `smp.metrics` with `reduction="macro-imagewise"`; final-epoch weights rather than best-validation weights; and ten test passes with independent random crops.

Two adaptations were unavoidable. The published notebook targets PyTorch Lightning 1.x, whose `training_epoch_end` and `validation_epoch_end` hooks were removed in 2.x; per-epoch aggregation was therefore reimplemented explicitly. The `A.Flip` transform has since been removed from `albumentations`; its semantics - selecting one of {vertical, horizontal, both} uniformly, rather than applying two independent flips - were reproduced explicitly and verified empirically over 4,000 samples, yielding 0.752 no-flip and 0.089 / 0.084 / 0.075 for the three flip modes against an expected 0.750 / 0.083 each.

A.3 Result and the batch-size constraint

Under this configuration the reproduction attained an IoU of 0.4470 ± 0.0059 under the reference reduction, against the published 0.7417 ± 0.0038 .

The discrepancy is attributable to GPU memory. The reference configuration trains at batch size 32. Measured peak allocation for MA-Net/EfficientNet-B3 at 256×256 in fp32 is 2.44 GB at batch 8, 4.65 GB at batch 16, and out-of-memory at batch 32 on a 4 GB card; at half precision, batch 32 requires 4.94 GB and therefore also does not fit. Batch 32 is thus unreachable on this hardware at any precision. The reproduction used batch 8 with gradient accumulation $\times 4$, which preserves the effective batch - and hence the gradient signal - but leaves batch normalisation estimating its statistics from 8 samples per forward pass rather than 32.

EfficientNet is batch-normalisation dense, and the degradation of BN statistics at small batch is a well-characterised effect; it is the motivating problem for Group Normalization [41, 19]. That this is the operative cause rather than a defect in the port is corroborated by the cluster measurements of Section 3.4.4, where the same code at batch 32 on a V100 attains 68–73% IoU, consistent with the published figure.

Consequence for this thesis. All accuracy figures reported in the main text are drawn from the cluster. The workstation environment is used for development, defect analysis and instrumentation work, for which it is adequate, and not for any claim about segmentation quality.

A.4 Closing the gap: full-protocol replication on the cluster

The batch-size hypothesis of Section A.3 was tested directly rather than left as an inference from BatchNorm theory. A from-scratch replica, **0-reproduction**, was built independently of both the workstation port above and the main PADS-Loader pipeline evaluated in Sections 5 and 6, for the specific purpose of isolating implementation differences from a genuine reproducibility gap. It was run on the cluster at the reference’s full protocol: batch size 32, 20 epochs with no early stopping, final-epoch weights, and ten independent test-set passes – for all three architectures used elsewhere in this thesis, not MA-Net alone.

Architecture	Test IoU (mean, 10 passes)	Range
DeepLabV3+	0.728	0.722–0.732
SegFormer	0.733	0.725–0.739
MA-Net	0.735	0.729–0.742
Reference (Casini et al., Model 1)	0.7417	± 0.0038

Table 21: Full-protocol replica results, all three architectures, against the reference figure. Standard deviation across the ten test passes was 0.003–0.004 for each architecture – tight enough that the result is not an artefact of a favourable single run.

All three architectures land within 1–1.4 percentage points of the reference figure. This closes the gap the workstation reproduction could not: the earlier result (0.4470, then 0.4504 under the reference’s own metric convention – see Section A.5) was constrained by batch size 8, and the diagnosis in Section A.3 is confirmed directly rather than only by analogy to published BatchNorm sensitivity results. It also settles a question that diagnosis alone could not: whether a correct batch size on its own would be sufficient, or whether some other, unidentified defect was also contributing. It was sufficient – once batch size, the full epoch budget, and final-epoch (rather than early-stopped) weights were all restored together, no further correction was needed to reach reference-comparable accuracy on any of the three architectures.

This result is the basis for the caveat in Section 5.7: the main PADS-Loader pipeline’s IoU figures throughout Sections 5 and 6 (clustering around 0.32–0.40) are not this workload’s ceiling. They reflect a pipeline that, independently of the batch-size question addressed here, differs from both the reference and this appendix’s own replica in two further respects: an IoU averaging convention that pools confusion-matrix counts across the validation split rather than averaging per-image (Section A.5 quantifies a 3.7-point effect from this alone on one checkpoint), and a flip-augmentation implementation that applies two independent single-axis flips rather than the reference’s **A.Flip** semantics (Appendix B, Table 25). Both are properties of `3-pads/*/model.py` and `3-pads/*/dataset.py` specifically, not of this appendix’s already-faithful port or of **0-reproduction**; neither pipeline used to produce Table 21 exhibits either defect. The two effects are not individually large enough to explain the full gap between ~ 0.35 and 0.73–0.74 on their own, and combined with an abbreviated training protocol (early stopping rather than a fixed 20 epochs, in the main pipeline’s evaluation runs) they account for the remainder.

A.5 Metric reduction

The reduction convention accounts for a material part of the difference between naively reported and reference-comparable numbers. Measured on the same trained checkpoint (Table 22):

Two hypotheses about the metric were tested and rejected. Excluding the empty tiles entirely changes pooled IoU by 0.001, because they contribute only 0.8% of false positives – the model classifies 102 of 104 empty tiles perfectly. Sweeping the decision threshold over $\{0.2, \dots, 0.7\}$

Reduction	IoU
Globally pooled, site-bearing tiles only	0.4131
Per-image mean, site-bearing tiles only	0.3364
Per-image mean, all tiles (reference convention)	0.4504

Table 22: Effect of the aggregation convention on reported IoU for an identical model. The reference convention counts a correctly-predicted empty tile as a perfect match.

confirms 0.5 is optimal, so the reported figure is not an artefact of a poorly chosen operating point.

A.6 Error structure

The aggregate figure conceals a strongly bimodal error distribution. On the validation split, per-image IoU has a 90th percentile of 0.785 - at reference level - while 35.3% of tiles score below 0.1. Performance is strongly monotonic in the fraction of the crop occupied by the site (Table 23).

Site area in crop	Tiles	Mean IoU	Fraction < 0.1
0.5–2%	25	0.024	0.92
2–5%	83	0.179	0.57
5–12%	137	0.285	0.37
12–30%	151	0.440	0.19
30–100%	86	0.488	0.23

Table 23: Per-image IoU by the fraction of the cropped tile occupied by the annotated site.

This is consistent with the original authors’ rationale for excluding sites below 1,000 m²: on small objects, a few pixels of boundary error destroys the ratio, and such annotations frequently denote sites of imprecise location rather than precise small mounds. It also explains the +3.9 point gain the reference reports from filtering (Model 1 → Model 2).

A.7 Accuracy guardrail: segmentation quality across data-path policies

The chapters on throughput report GPU utilisation only, because throughput is what PADS-Loader sets out to change. That leaves a separate question which those chapters state but do not evidence: whether any of the data-path configurations degrades what the model actually learns. Sharding, staging and prefetching each alter the storage format, the ordering, or the materialisation time of training samples, and a defect in any of them could silently corrupt, duplicate or stale-serve data. A throughput gain obtained that way would be worthless, so segmentation quality is measured for every configuration and reported here.

The result is a negative one, which is what was wanted. Within each architecture, quality varies by at most ± 2.5 percentage points across all four policies, and the variation has no consistent direction: SegFormer scores highest under full PADS-Loader and lowest under `loose`, while DeepLabV3+ does the reverse. There is no systematic accuracy cost to sharding or staging in this data, and equally no accuracy benefit – the data path moves throughput, and leaves what the model learns alone.

Two caveats bound how far this evidence reaches. First, every value in Table 24 is a single run ($n = 1$), so a ± 2.5 point spread cannot be separated from run-to-run variance that this evaluation never measured (Section 8). The claim supported is the absence of a large, systematic, one-directional degradation, not the absence of any effect. Second, these figures are depressed relative to the reference by the two defects of Appendix B – the metric-averaging convention and the flip-augmentation error – which apply identically to every run in the table. That uniformity is what makes the comparison between policies valid while leaving the absolute values unrepresentative of the architectures’ capability; Section A.4 establishes what that capability actually is.

Architecture	Policy	IoU (pooled)
DeepLabV3+	none (loose)	0.3707
	shard only	0.3569
	stage only	0.3582
	full PADS-Loader (adaptive)	0.3459
MA-Net	none (loose)	0.3695
	shard only	0.3527
	stage only	0.3602
	full PADS-Loader (adaptive)	0.3536
SegFormer	none (loose)	0.3647
	shard only	0.3735
	stage only	0.3689
	full PADS-Loader (adaptive)	0.3757

Table 24: Segmentation quality by data-path policy, for the same runs whose throughput is reported in Section 5, Table 12 and Section 6, Table 18. IoU here is the main-pipeline pooled convention under `torchmetrics.BinaryJaccardIndex`, not the reference-comparable per-image mean of Section A.5: these values are comparable to one another but not to Table 2.

A.8 Qualitative output

Figure 1, shown in Section 1, gives the qualitative view for these checkpoints on held-out tiles. This section states how those examples were chosen and how they sit relative to the rest of the test set.

Where the models succeed. Scoring all 466 site-bearing test tiles under all three checkpoints, roughly a third score above 0.5 and the best-performing tenth reach 0.78–0.81. On these tiles the predicted contour tracks the annotated boundary closely rather than merely overlapping its extent, and the three architectures agree with one another to within a few points. This is a real capability: on a substantial minority of sites the models delineate the mound accurately enough to be directly useful to the survey workflow the reference study describes.

Where they fail, and how the examples were selected. The distribution is bimodal rather than centred: alongside the successes above, 48–57% of site-bearing tiles score below 0.1, and median per-tile IoU is 0.165 (MA-Net), 0.053 (DeepLabV3+) and 0.000 (SegFormer). The models tend either to find a site and outline it well or to miss it almost entirely. Figure 1’s examples are therefore selected rather than randomly sampled – three from the upper mode, one characteristic failure, one negative – because a random draw of five tiles would sample mostly from the lower mode and represent the successful regime no better than the failing one.

The failure mode is systematic and is the one Section A.6 quantifies. Taking the worst of the three architectures per tile, median IoU is 0.000 for every site-area band below 12% of the crop and rises only for larger sites. Small sites carry too little tonal evidence to separate them from the terrain around them, which is the same reason the original authors excluded sites below 1,000 m² and report a +3.9 point gain from that filtering.

Negatives. On tiles containing no annotated site, all three architectures correctly predict nothing in the large majority of cases: 108 of the 123 negative test tiles are cleanly rejected by all three, with spurious detections on only 5.7% (DeepLabV3+), 7.3% (MA-Net) and 8.9% (SegFormer). This bears on the metric discussion of Section A.5: correctly-rejected empty tiles score a perfect 1.0 under the reference’s per-image convention and so contribute materially to the headline figure, and the fact that they are genuinely rejected – rather than scored as perfect by an artefact of averaging – is what makes that contribution legitimate.

A note on per-tile figures. Evaluation random-crops each tile, following the reference protocol, so a single tile’s score depends on the crop drawn as well as on the model. The values annotated in Figure 1 are those of the crop shown; the distributional statistics above, taken over all 466 site-bearing tiles, are the stable quantity. This appendix documents the reproduction environment, protocol, metric reduction, and qualitative error analysis needed to interpret the accuracy guardrail used in the main chapters.

B Pipeline Defects Found and Corrected

Porting the training code to a second environment surfaced a number of defects in the experimental pipeline. This appendix records them, what each one did to the measurements, and how it was resolved.

Two things are worth stating at the outset, because the length of the list invites a misreading. First, almost all of these defects are *silent*: they degrade results or invalidate measurements without raising an error, which is precisely why they survive casual testing and why porting to a second environment is an effective way to find them. A pipeline that produced plausible numbers throughout was, in several respects, not measuring what it reported. Second, all but the few explicitly identified below as outstanding were fixed, and every result reported in Sections 5 and 6 was collected after those fixes were in place – the evaluation sweep post-dates the corrections by a week. This appendix is therefore a record of defects found and closed, not a list of caveats attaching to the results.

It is included because reproduction is the mechanism that surfaced these problems, and because the same defects are the sort that any reimplementing of this workload would be liable to repeat. Documenting them is more useful to a reader attempting that than a clean narrative would be.

B.1 Staging mechanism: three defects, all corrected

B.1.1 Policy selected after the datasets it selects were constructed

The staging policy was applied only to `DataLoader` construction, not to dataset construction. In the original control flow, the three datasets were built before the profiler ran:

```

1 policy = base_config.get("data_policy", "loose") # key absent -> "loose"
2 train_dataset = create_dataset(..., policy=policy) # always the loose reader
3 ...
4 base_config["data_policy"] = policy_summary["policy"] # profiler decides, too
   late
5 train_loader = build_data_loader(train_dataset, policy=base_config["data_policy"])

```

The datasets were never rebuilt after the profiler had spoken, and the selected policy reached only `build_data_loader`, whose entire use of it is to choose `num_workers` and `prefetch_factor`. The shard and stage reader classes were therefore unreachable code, and every run - irrespective of the policy the profiler reported - read loose files.

The fix reorders policy selection ahead of dataset construction: the profiler probes an explicitly-loose dataset, and the working datasets are then built from its verdict, so the selected reader is the one actually used. Staging results collected before this change describe a policy *decision* rather than a policy *execution*, and were superseded by the re-run campaign reported in this thesis.

B.1.2 No archive the shard reader could open

The shard reader addresses members by original filename within archives named `originals.tar` and `negs.tar`. The repository’s shard generator produces WebDataset-format shards (`shard-000000.tar`, members keyed `train_000123.jpg`), which that reader can neither open nor address; and no archives of either form existed on disk. The shard branch would therefore have fallen through to its loose fallback even had it been reachable.

Archives in the required layout are now produced by a dedicated builder, which verifies its output by comparing decoded samples against the loose path. They are written uncompressed for

the reason given in Section 4: the payload is already-compressed JPEG and PNG, so compression yields negligible size reduction while forcing decompression from the stream start on every random access.

B.1.3 Staging silently degraded to a plain read

The staging dataset copied samples to a scratch directory while swallowing `IOError` and `OSError`. Because the scratch directory was never created, every copy failed silently and the reader fell back to the source path, appearing selected and active while having no effect. The directory is now created up front and copy failures are surfaced rather than absorbed.

B.2 Issues bearing on segmentation accuracy

B.3 A measurement consideration: profiling perturbs the training RNG

One further interaction is worth recording, less as a defect than as a constraint on how two runs may be compared. The warm-up pass iterates the *training* dataset, whose augmentation transforms are stochastic, and so advances the global random state before training proper begins. A profiled run is therefore not seed-identical to an unprofiled one at otherwise identical configuration.

The effect is small and was quantified rather than assumed: repeated runs differ by approximately ± 0.02 IoU, which bounds the comparison. Either the random state is restored after profiling, or differences below that magnitude between a profiled and an unprofiled run are treated as run-to-run variance rather than as an effect of profiling. The latter convention is used throughout this thesis.

A small number of purely operational issues encountered during the work – checkpoint file-names interpolating a metric name containing a path separator, and hyperparameter studies not persisting across an interrupted sweep – are omitted here. They cost time but have no bearing on the validity of any measurement reported.

B.4 Further issues found during the evaluation campaign

Four further defects were identified while running the full evaluation sweep. The first two are infrastructure failures rather than correctness bugs, but are documented because they caused real job losses and shaped the retry pattern visible in some of the job identifiers referenced in Sections 5 and 6.

- **Scratch directory scoped to `project_root`, not shared.** `resolve_scratch_dir()` derived the node-local staging path from each job’s `project_root`, which is deliberately unique per experiment configuration for checkpoint isolation. Because every `stage-policy` job across the evaluation sweep therefore had its own scratch directory, each of the roughly forty such jobs independently copied its own multi-gigabyte snapshot of the dataset, accumulating 69 redundant copies (≈ 91 GB) and exhausting the account’s disk quota mid-sweep, which in turn failed every subsequent job attempting to write output. A real node-local scratch mount is shared per node regardless of which job is using it; the fix decouples `scratch_dir` from `project_root` and defaults to one fixed, shared location unless explicitly overridden.
- **Checkpoint auto-resume from a truncated file.** Several jobs that had progressed far enough to begin writing `last.ckpt` before the disk quota was exhausted left a truncated checkpoint behind. Lightning resumes training from `last.ckpt` by default if one is present in the checkpoint directory, and loading a truncated `.ckpt` raises `RuntimeError: PytorchStreamReader failed reading zip archive` rather than falling back to a fresh run. Affected job directories were identified and their partial checkpoints removed before resubmission.
- **0-reproduction’s epoch-end hooks never wired in for two of three architectures.** `build_model()` and `load_model_from_checkpoint()` instantiated the plain `ArcheoModel`

Defect	Description and evidence
Focal loss α unset in one variant	<code>smp.losses.FocalLoss</code> defaults to <code>alpha=None</code> , applying no class weighting on a target with a median 7% positive-pixel rate.
Validation on a random crop	Scoring a different random patch of each tile every epoch injects variance into checkpoint selection and early stopping. Note that the reference <i>deliberately</i> random-crops at test time and averages ten passes; the defect is using it as an unaveraged per-epoch selection signal.
Configured loss silently ignored	One model hardcoded <code>BCEWithLogitsLoss</code> while both the configuration and the training log reported <code>focal</code> . Correcting it moved epoch-1 IoU from 0.299 to 0.362 and reversed a declining validation trend. Unknown loss names now raise rather than falling through.
fp16 divergence on EfficientNet	Under <code>16-mixed</code> , training loss became non-finite at epoch 4 and all metrics collapsed to zero, while early stopping continued for its full 15-epoch patience on a dead model; the run reported a “best” score that was merely the last value before the blowup. ResNet-50 was unaffected across 20 epochs. Isolating the loss under <code>autocast</code> showed gradients bit-identical to fp32, locating the overflow in the encoder - consistent with EfficientNet’s SiLU and squeeze-excite blocks exceeding fp16’s exponent range. Resolved by <code>bf16-mixed</code> ; a guard now aborts at the epoch a loss becomes non-finite.
IoU averaging convention (main pipeline)	<code>3-pads/*/model.py</code> computes validation/test IoU with <code>torchmetrics.BinaryJaccardIndex</code> , which pools confusion-matrix counts across the whole split before dividing, rather than the reference’s per-image mean (Section A.5, which quantifies a 3.7-point effect from this convention alone on one checkpoint). Not present in either the Appendix A workstation port or 0-reproduction, both of which use <code>smp.metrics.iou_score</code> with <code>reduction="macro-imagewise"</code> throughout.
Flip-augmentation semantics (main pipeline)	<code>3-pads/*/dataset.py</code> applies two independent single-axis flip transforms rather than the reference’s <code>A.Flip</code> , which selects one of {vertical, horizontal, both} uniformly (Appendix A, Section A.2 measures the reference semantics empirically at 0.752 no-flip and 0.089/0.084/0.075 for the three flip modes, against an expected 0.750/0.083 each). Changes the effective augmentation distribution seen during training; not quantified in isolation against IoU, since it is confounded with the metric-convention defect above in every main-pipeline run collected this session.

Table 25: Issues bearing on segmentation accuracy. The first four were corrected. The final two are known differences between the main pipeline and the reference convention, retained deliberately so that every run in this thesis is measured identically; they are specific to `3-pads/*/model.py` and `3-pads/*/dataset.py` (Sections 5 and 6’s pipeline); neither is present in Appendix A’s workstation port or in 0-reproduction (Section A.4).

base class for the MA-Net and DeepLabV3+ code paths, rather than `_EpochBufferedArcheoModel`, the subclass that implements `on_{train,validation,test}_epoch_end` in place of the Lightning 1.x `*_epoch_end(self, outputs)` hooks the ported notebook code assumes (Lightning 2.x removed these; see also Section A.2). Only the SegFormer path used the correct subclass. Training completed without error under the plain base class – per-step loss

logging does not depend on the missing hook – so the defect was silent until the test loop’s aggregated metric lookup raised `KeyError: 'test/IOU-img'`, since the epoch-aggregated IoU had never actually been logged. Both non-SegFormer paths now use the buffered subclass; this was confirmed by a full run reaching the expected metric before the Table 21 results were collected.

- **Profiler warm-up window bias – implementation diverged from its own documented design.** Documented in full in Section 5.6 rather than repeated here: the default 8-batch profiling window included one-time CUDA/cuDNN warm-up cost in its T_{gpu} average, inflating it 6–10 $\times$ and systematically suppressing the adaptive policy across most of Section 6’s threshold sweep. The design had anticipated this failure mode: Section 4 specifies discarding the first $w_0 = 10$ batches and aggregating by median rather than mean, for precisely this reason. The divergence was between that specification and the shipped `profile_data_pipeline()`, which implemented neither safeguard – which is also why the fix was straightforward once the symptom was traced. Discard is now implemented and validated (`-sweep_discard`), recovering full-epoch measurement accuracy at negligible cost. Median aggregation remains untested: a median is inherently less sensitive than a mean to a small number of outlying warm-up samples, so it may add robustness independent of discarding, but that has not been measured and is not claimed here. Included in this list because, unlike the first three defects, it is a correctness defect in PADS-Loader itself rather than in the evaluation infrastructure, and it directly affects how several results in Sections 5 and 6 should be interpreted.

B.5 Implication for the results reported in this thesis

Section B.1.1 and the missing archives were the consequential defects. Because the selected policy never reached the data path, and because the shard policy had no readable data in any case, any staging result collected before those fixes describes a policy *decision* rather than a policy *execution*. Such results are not recoverable by reanalysis; the experiments had to be re-run.

They were. Both defects were corrected before the evaluation campaign reported in this thesis was carried out, and the full sweep was then collected against the fixed pipeline. Every figure in Sections 5 and 6 therefore describes policies that actually executed, with the shard and stage readers reachable and supplied with archives in a layout they can open. The measurements that motivated this appendix have been superseded rather than merely annotated, and no result presented in this thesis rests on them.

One qualification belongs here rather than in the main text. That the staging policies now execute does not mean the adaptive layer routinely selects them: Section 5.6 establishes that the profiler’s warm-up measurement was biased in a way that suppressed staging across most of the threshold sweep, which is a separate defect with a separate resolution. The distinction matters for reading Section 6: where a run reports `loose`, that is now a decision the profiler genuinely made on the evidence available to it, not an artefact of the selected policy failing to reach the data path.

Finally, these defects are properties of the code rather than of any particular machine, and applied equally to workstation and cluster runs. That is what makes them worth documenting: none would have been caught by running the same code again on the same hardware. This appendix records defects discovered during reproduction and evaluation, distinguishing corrections that affect staging behaviour from those that affect segmentation accuracy or measurement validity.